NZSE 22

Generated by Doxygen 1.9.4

1 Namespace Index 2
1.1 Package List . . . 2

2 Hierarchical Index 2
2.1 Class Hierarchy . . . 2

3 Class Index 3
3.1 Class List . . . 3

4 File Index 3
4.1 File List . . . 3

5 Namespace Documentation 4
5.1 Package de.hda.nzse22 . . . 4
5.2 Package de.hda.nzse22.download . . . 4
5.3 Package de.hda.nzse22.favoritesAdapter . . . 4
5.4 Package de.hda.nzse22.mapAdapter . . . 4
5.5 Package de.hda.nzse22.model . . . 4

6 Class Documentation 5
6.1 de.hda.nzse22.model.chargingStation Class Reference . . . 5
6.1.1 Detailed Description . . . 7
6.1.2 Constructor & Destructor Documentation . . . 7
6.1.3 Member Function Documentation . . . 7
6.1.4 Member Data Documentation . . . 11
6.2 de.hda.nzse22.model.ChargingStationDAO Interface Reference . . . 14
6.2.1 Detailed Description . . . 14
6.2.2 Member Function Documentation . . . 14
6.3 de.hda.nzse22.FavoriteActivity Class Reference . . . 16
6.3.1 Detailed Description . . . 17
6.3.2 Member Function Documentation . . . 17
6.4 de.hda.nzse22.favoritesAdapter.favoritesAdapter Class Reference . . . 18
6.4.1 Detailed Description . . . 19
6.4.2 Constructor & Destructor Documentation . . . 19
6.4.3 Member Function Documentation . . . 20
6.5 de.hda.nzse22.download.GetChargingStationsActivity Class Reference . . . 21
6.5.1 Detailed Description . . . 22
6.5.2 Member Function Documentation . . . 22
6.6 de.hda.nzse22.MainActivity Class Reference . . . 25
6.6.1 Detailed Description . . . 26
6.6.2 Member Function Documentation . . . 26
6.7 de.hda.nzse22.MapActivity Class Reference . . . 27
6.7.1 Detailed Description . . . 28
6.7.2 Member Function Documentation . . . 28

6.8 de.hda.nzse22.mapAdapter.mapAdapter Class Reference . . . 29
6.8.1 Detailed Description . . . 30
6.8.2 Constructor & Destructor Documentation . . . 30
6.8.3 Member Function Documentation . . . 31
6.9 de.hda.nzse22.model.NZSEDatabase Class Reference . . . 32
6.9.1 Detailed Description . . . 33
6.9.2 Member Function Documentation . . . 33
6.10 de.hda.nzse22.download.RequestService Class Reference . . . 35
6.10.1 Detailed Description . . . 37
6.10.2 Constructor & Destructor Documentation . . . 37
6.10.3 Member Function Documentation . . . 37
6.10.4 Member Data Documentation . . . 39
6.11 de.hda.nzse22.download.RequestService.RequestServiceBinder Class Reference . . . 41
6.11.1 Detailed Description . . . 42
6.11.2 Member Function Documentation . . . 42

7 File Documentation **43**
7.1 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/GetChargingStationsActivity.java File Reference . . . 43
7.2 GetChargingStationsActivity.java . . . 43
7.3 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/RequestService.java File Reference . . . 46
7.4 RequestService.java . . . 46
7.5 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/FavoriteActivity.java File Reference . . . 48
7.6 FavoriteActivity.java . . . 48
7.7 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/favoritesAdapter/favoritesAdapter.java File Reference 51
7.8 favoritesAdapter.java . . . 51
7.9 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MainActivity.java File Reference . . . 53
7.10 MainActivity.java . . . 53
7.11 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MapActivity.java File Reference . . . 56
7.12 MapActivity.java . . . 57
7.13 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/mapAdapter/mapAdapter.java File Reference . . . 59
7.14 mapAdapter.java . . . 59
7.15 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/chargingStation.java File Reference . . . 61
7.16 chargingStation.java . . . 61
7.17 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/ChargingStationDAO.java File Reference . . . 64
7.18 ChargingStationDAO.java . . . 64
7.19 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/NZSEDatabase.java File Reference . . . 64
7.20 NZSEDatabase.java . . . 65

Index **67**

1 Namespace Index

1.1 Package List

Here are the packages with brief descriptions (if available):

de.hda.nzse22 4

de.hda.nzse22.download 4

de.hda.nzse22.favoritesAdapter 4

de.hda.nzse22.mapAdapter 4

de.hda.nzse22.model 4

2 Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

RecyclerView.Adapter

de.hda.nzse22.favoritesAdapter.favoritesAdapter 18

de.hda.nzse22.mapAdapter.mapAdapter 29

de.hda.nzse22.model.chargingStation 5

de.hda.nzse22.model.ChargingStationDAO 14

AppCompatActivity

de.hda.nzse22.FavoriteActivity 16

de.hda.nzse22.MainActivity 25

de.hda.nzse22.MapActivity 27

de.hda.nzse22.download.GetChargingStationsActivity 21

Binder

de.hda.nzse22.download.RequestService.RequestServiceBinder 41

OnMapReadyCallback

de.hda.nzse22.MapActivity 27

RoomDatabase

de.hda.nzse22.model.NZSEDatabase 32

Service

de.hda.nzse22.download.RequestService 35

3 Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

de.hda.nzse22.model.chargingStation 5

de.hda.nzse22.model.ChargingStationDAO 14

de.hda.nzse22.FavoriteActivity 16

de.hda.nzse22.favoritesAdapter.favoritesAdapter 18

de.hda.nzse22.download.GetChargingStationsActivity 21

de.hda.nzse22.MainActivity 25

de.hda.nzse22.MapActivity 27

de.hda.nzse22.mapAdapter.mapAdapter 29

de.hda.nzse22.model.NZSEDatabase 32

de.hda.nzse22.download.RequestService 35

de.hda.nzse22.download.RequestService.RequestServiceBinder 41

4 File Index

4.1 File List

Here is a list of all files with brief descriptions:

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/FavoriteActivity.java 48

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MainActivity.java 53

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MapActivity.java 56

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/GetChargingStationsActivity.java 43

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/RequestService.java 46

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/favoritesAdapter/favoritesAdapter.java 51

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/mapAdapter/mapAdapter.java 59

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/chargingStation.java 61

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/ChargingStationDAO.java 64

D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/NZSEDatabase.java 64

5 Namespace Documentation

5.1 Package de.hda.nzse22

Packages

- package download
- package favoritesAdapter
- package mapAdapter
- package model

Classes

- class FavoriteActivity
- class MainActivity
- class MapActivity

5.2 Package de.hda.nzse22.download

Classes

- class GetChargingStationsActivity
- class RequestService

5.3 Package de.hda.nzse22.favoritesAdapter

Classes

- class favoritesAdapter

5.4 Package de.hda.nzse22.mapAdapter

Classes

- class mapAdapter

5.5 Package de.hda.nzse22.model

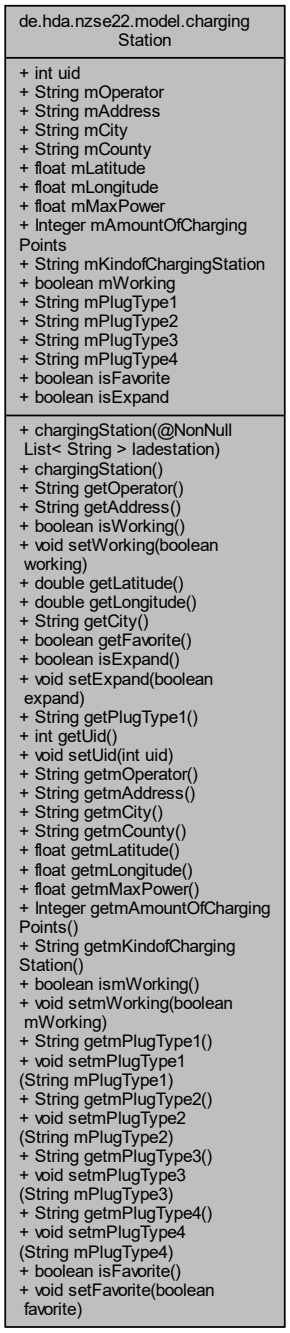
Classes

- class chargingStation
- interface ChargingStationDAO
- class NZSEDatabase

6 Class Documentation

6.1 de.hda.nzse22.model.chargingStation Class Reference

Collaboration diagram for de.hda.nzse22.model.chargingStation:



Public Member Functions

- chargingStation (@NonNull List< String > ladestation)

- chargingStation ()
- String getOperator ()
- String getAddress ()
- boolean isWorking ()
- void setWorking (boolean working)
- double getLatitude ()
- double getLongitude ()
- String getCity ()
- boolean getFavorite ()
- boolean isExpand ()
- void setExpand (boolean expand)
- String getPlugType1 ()
- int getUid ()
- void setUid (int uid)
- String getmOperator ()
- String getmAddress ()
- String getmCity ()
- String getmCounty ()
- float getmLatitude ()
- float getmLongitude ()
- float getmMaxPower ()
- Integer getmAmountOfChargingPoints ()
- String getmKindofChargingStation ()
- boolean ismWorking ()
- void setmWorking (boolean mWorking)
- String getmPlugType1 ()
- void setmPlugType1 (String mPlugType1)
- String getmPlugType2 ()
- void setmPlugType2 (String mPlugType2)
- String getmPlugType3 ()
- void setmPlugType3 (String mPlugType3)
- String getmPlugType4 ()
- void setmPlugType4 (String mPlugType4)
- boolean isFavorite ()
- void setFavorite (boolean favorite)

Public Attributes

- int uid
- String mOperator = ""
- String mAddress = ""
- String mCity = ""
- String mCounty = ""
- float mLatitude = 0
- float mLongitude = 0
- float mMaxPower = 0
- Integer mAmountOfChargingPoints = 0
- String mKindofChargingStation = ""
- boolean mWorking = true
- String mPlugType1 = " "
- String mPlugType2 = " "
- String mPlugType3 = " "
- String mPlugType4 = " "
- boolean isFavorite = false
- boolean isExpand = false

6.1.1 Detailed Description

Definition at line 13 of file chargingStation.java.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 chargingStation() [1/2] de.hda.nzse22.model.chargingStation.chargingStation (@NonNull List< String > *ladestation*)

Definition at line 64 of file chargingStation.java.

6.1.2.2 chargingStation() [2/2] de.hda.nzse22.model.chargingStation.chargingStation ( )

Definition at line 86 of file chargingStation.java.

6.1.3 Member Function Documentation

6.1.3.1 getAddress() String de.hda.nzse22.model.chargingStation.getAddress ( )

Definition at line 94 of file chargingStation.java.

6.1.3.2 getCity() String de.hda.nzse22.model.chargingStation.getCity ( )

Definition at line 114 of file chargingStation.java.

6.1.3.3 getFavorite() boolean de.hda.nzse22.model.chargingStation.getFavorite ( )

Definition at line 118 of file chargingStation.java.

6.1.3.4 getLatitude() double de.hda.nzse22.model.chargingStation.getLatitude ( )

Definition at line 106 of file chargingStation.java.

6.1.3.5 getLongitude() `double de.hda.nzse22.model.chargingStation.getLongitude ( )`

Definition at line 110 of file chargingStation.java.

6.1.3.6 getmAddress() `String de.hda.nzse22.model.chargingStation.getmAddress ( )`

Definition at line 146 of file chargingStation.java.

6.1.3.7 getmAmountOfChargingPoints() `Integer de.hda.nzse22.model.chargingStation.getmAmount↩ OfChargingPoints ( )`

Definition at line 170 of file chargingStation.java.

6.1.3.8 getmCity() `String de.hda.nzse22.model.chargingStation.getmCity ( )`

Definition at line 150 of file chargingStation.java.

6.1.3.9 getmCounty() `String de.hda.nzse22.model.chargingStation.getmCounty ( )`

Definition at line 154 of file chargingStation.java.

6.1.3.10 getmKindofChargingStation() `String de.hda.nzse22.model.chargingStation.getmKindof↩ ChargingStation ( )`

Definition at line 174 of file chargingStation.java.

6.1.3.11 getmLatitude() `float de.hda.nzse22.model.chargingStation.getmLatitude ( )`

Definition at line 158 of file chargingStation.java.

6.1.3.12 getmLongitude() `float de.hda.nzse22.model.chargingStation.getmLongitude ( )`

Definition at line 162 of file chargingStation.java.

6.1.3.13 getmMaxPower() `float de.hda.nzse22.model.chargingStation.getmMaxPower ( )`

Definition at line 166 of file chargingStation.java.

6.1.3.14 getmOperator() `String de.hda.nzse22.model.chargingStation.getmOperator ( )`

Definition at line 142 of file chargingStation.java.

6.1.3.15 getmPlugType1() `String de.hda.nzse22.model.chargingStation.getmPlugType1 ( )`

Definition at line 186 of file chargingStation.java.

6.1.3.16 getmPlugType2() `String de.hda.nzse22.model.chargingStation.getmPlugType2 ( )`

Definition at line 194 of file chargingStation.java.

6.1.3.17 getmPlugType3() `String de.hda.nzse22.model.chargingStation.getmPlugType3 ( )`

Definition at line 202 of file chargingStation.java.

6.1.3.18 getmPlugType4() `String de.hda.nzse22.model.chargingStation.getmPlugType4 ( )`

Definition at line 210 of file chargingStation.java.

6.1.3.19 getOperator() `String de.hda.nzse22.model.chargingStation.getOperator ( )`

Definition at line 90 of file chargingStation.java.

6.1.3.20 getPlugType1() `String de.hda.nzse22.model.chargingStation.getPlugType1 ( )`

Definition at line 130 of file chargingStation.java.

6.1.3.21 getUid() `int de.hda.nzse22.model.chargingStation.getUid ( )`

Definition at line 134 of file chargingStation.java.

6.1.3.22 isExpand() `boolean de.hda.nzse22.model.chargingStation.isExpand ( )`

Definition at line 122 of file chargingStation.java.

6.1.3.23 isFavorite() `boolean de.hda.nzse22.model.chargingStation.isFavorite ( )`

Definition at line 218 of file chargingStation.java.

6.1.3.24 ismWorking() `boolean de.hda.nzse22.model.chargingStation.ismWorking ( )`

Definition at line 178 of file chargingStation.java.

6.1.3.25 isWorking() `boolean de.hda.nzse22.model.chargingStation.isWorking ( )`

Definition at line 98 of file chargingStation.java.

6.1.3.26 setExpand() `void de.hda.nzse22.model.chargingStation.setExpand ( boolean expand )`

Definition at line 126 of file chargingStation.java.

6.1.3.27 setFavorite() `void de.hda.nzse22.model.chargingStation.setFavorite ( boolean favorite )`

Definition at line 222 of file chargingStation.java.

6.1.3.28 setmPlugType1() `void de.hda.nzse22.model.chargingStation.setmPlugType1 ( String mPlugType1 )`

Definition at line 190 of file chargingStation.java.

6.1.3.29 setmPlugType2()

```
void de.hda.nzse22.model.chargingStation.setmPlugType2 (
    String mPlugType2 )
```

Definition at line 198 of file chargingStation.java.

6.1.3.30 setmPlugType3()

```
void de.hda.nzse22.model.chargingStation.setmPlugType3 (
    String mPlugType3 )
```

Definition at line 206 of file chargingStation.java.

6.1.3.31 setmPlugType4()

```
void de.hda.nzse22.model.chargingStation.setmPlugType4 (
    String mPlugType4 )
```

Definition at line 214 of file chargingStation.java.

6.1.3.32 setmWorking()

```
void de.hda.nzse22.model.chargingStation.setmWorking (
    boolean mWorking )
```

Definition at line 182 of file chargingStation.java.

6.1.3.33 setUid()

```
void de.hda.nzse22.model.chargingStation.setUid (
    int uid )
```

Definition at line 138 of file chargingStation.java.

6.1.3.34 setWorking()

```
void de.hda.nzse22.model.chargingStation.setWorking (
    boolean working )
```

Definition at line 102 of file chargingStation.java.

6.1.4 Member Data Documentation

6.1.4.1 isExpand

```
boolean de.hda.nzse22.model.chargingStation.isExpand = false
```

Definition at line 62 of file chargingStation.java.

6.1.4.2 isFavorite `boolean de.hda.nzse22.model.chargingStation.isFavorite = false`

Definition at line 60 of file chargingStation.java.

6.1.4.3 mAddress `String de.hda.nzse22.model.chargingStation.mAddress = ""`

Definition at line 21 of file chargingStation.java.

6.1.4.4 mAmountOfChargingPoints `Integer de.hda.nzse22.model.chargingStation.mAmountOf↩ ChargingPoints = 0`

Definition at line 39 of file chargingStation.java.

6.1.4.5 mCity `String de.hda.nzse22.model.chargingStation.mCity = ""`

Definition at line 24 of file chargingStation.java.

6.1.4.6 mCounty `String de.hda.nzse22.model.chargingStation.mCounty = ""`

Definition at line 27 of file chargingStation.java.

6.1.4.7 mKindofChargingStation `String de.hda.nzse22.model.chargingStation.mKindofCharging↩ Station = ""`

Definition at line 42 of file chargingStation.java.

6.1.4.8 mLatitude `float de.hda.nzse22.model.chargingStation.mLatitude = 0`

Definition at line 30 of file chargingStation.java.

6.1.4.9 mLongitude `float de.hda.nzse22.model.chargingStation.mLongitude = 0`

Definition at line 33 of file chargingStation.java.

6.1.4.10 mMaxPower `float de.hda.nzse22.model.chargingStation.mMaxPower = 0`

Definition at line 36 of file chargingStation.java.

6.1.4.11 mOperator `String de.hda.nzse22.model.chargingStation.mOperator = ""`

Definition at line 18 of file chargingStation.java.

6.1.4.12 mPlugType1 `String de.hda.nzse22.model.chargingStation.mPlugType1 = " "`

Definition at line 48 of file chargingStation.java.

6.1.4.13 mPlugType2 `String de.hda.nzse22.model.chargingStation.mPlugType2 = " "`

Definition at line 51 of file chargingStation.java.

6.1.4.14 mPlugType3 `String de.hda.nzse22.model.chargingStation.mPlugType3 = " "`

Definition at line 54 of file chargingStation.java.

6.1.4.15 mPlugType4 `String de.hda.nzse22.model.chargingStation.mPlugType4 = " "`

Definition at line 57 of file chargingStation.java.

6.1.4.16 mWorking `boolean de.hda.nzse22.model.chargingStation.mWorking = true`

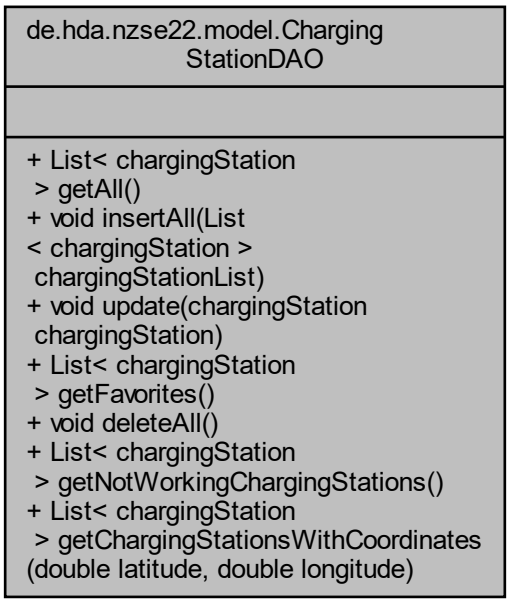
Definition at line 45 of file chargingStation.java.

6.1.4.17 uid `int de.hda.nzse22.model.chargingStation.uid`

Definition at line 15 of file chargingStation.java.

6.2 de.hda.nzse22.model.ChargingStationDAO Interface Reference

Collaboration diagram for de.hda.nzse22.model.ChargingStationDAO:



Public Member Functions

- List< chargingStation > getAll ()
- void insertAll (List< chargingStation > chargingStationList)
- void update (chargingStation chargingStation)
- List< chargingStation > getFavorites ()
- void deleteAll ()
- List< chargingStation > getNotWorkingChargingStations ()
- List< chargingStation > getChargingStationsWithCoordinates (double latitude, double longitude)

6.2.1 Detailed Description

Definition at line 11 of file ChargingStationDAO.java.

6.2.2 Member Function Documentation

6.2.2.1 deleteAll() `void de.hda.nzse22.model.ChargingStationDAO.deleteAll ( )`

Deletes all entities of the database

6.2.2.2 getAll()

```
List< chargingStation > de.hda.nzse22.model.ChargingStationDAO.getAll ( )
```

Returns a list of all chargingstations in the database

Returns

List of all chargingstations

6.2.2.3 getChargingStationsWithCoordinates()

```
List< chargingStation > de.hda.nzse22.model.↩
ChargingStationDAO.getChargingStationsWithCoordinates (
            double latitude,
            double longitude )
```

Returns a list of chargingstations with the given coordinates

Returns

List of chargingstations with the given coordinates

6.2.2.4 getFavorites()

```
List< chargingStation > de.hda.nzse22.model.ChargingStationDAO.get↩
Favorites ( )
```

Returns a list of all favorite chargingstations

Returns

List of all favorite chargingstations

6.2.2.5 getNotWorkingChargingStations()

```
List< chargingStation > de.hda.nzse22.model.Charging↩
StationDAO.getNotWorkingChargingStations ( )
```

6.2.2.6 insertAll()

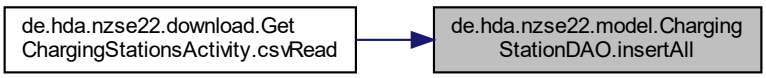
```
void de.hda.nzse22.model.ChargingStationDAO.insertAll (
            List< chargingStation > chargingStationList )
```

Inserts a list of chargingstations into the database

Parameters

chargingStationList	List of Chargingstations which are inserted into the database

Here is the caller graph for this function:



6.2.2.7 update() `void de.hda.nzse22.model.ChargingStationDAO.update ( chargingStation chargingStation )`

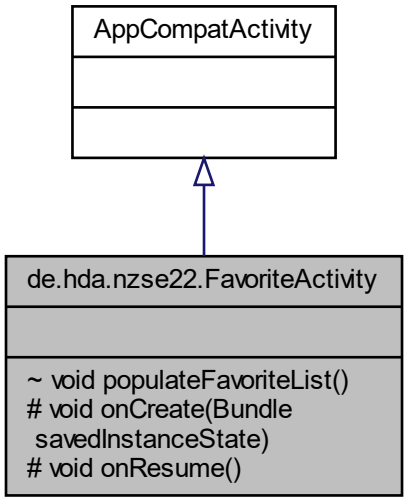
Updates given chargingstation object in the database

Parameters

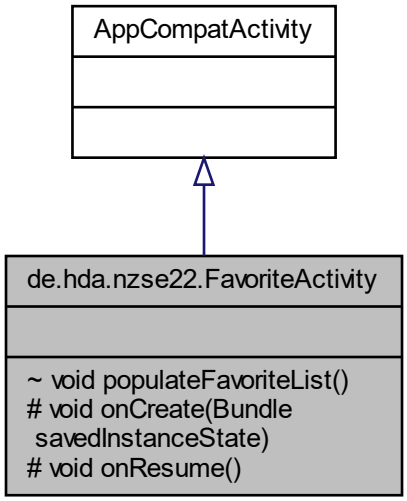
chargingStation	Chargingstation which will be updated in the databe

6.3 de.hda.nzse22.FavoriteActivity Class Reference

Inheritance diagram for de.hda.nzse22.FavoriteActivity:



Collaboration diagram for de.hda.nzse22.FavoriteActivity:



Protected Member Functions

- void onCreate (Bundle savedInstanceState)
- void onResume ()

6.3.1 Detailed Description

Definition at line 33 of file FavoriteActivity.java.

6.3.2 Member Function Documentation

6.3.2.1 onCreate() `void de.hda.nzse22.FavoriteActivity.onCreate ( Bundle savedInstanceState ) [protected]`

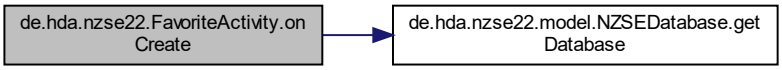
Creates the activity

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState(Bundle). (https://developer.android.com/reference/android/app/Activity#onCreate(android.os.Bundle))

Definition at line 57 of file FavoriteActivity.java.

Here is the call graph for this function:



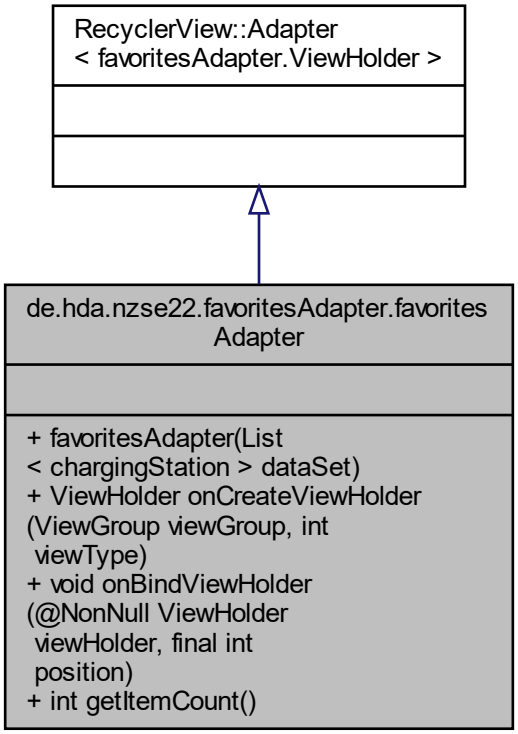
6.3.2.2 onResume() `void de.hda.nzse22.FavoriteActivity.onResume ( ) [protected]`

When activity is resumed, initialise NavigationBar, RecyclerView and reload favorite chargingstations

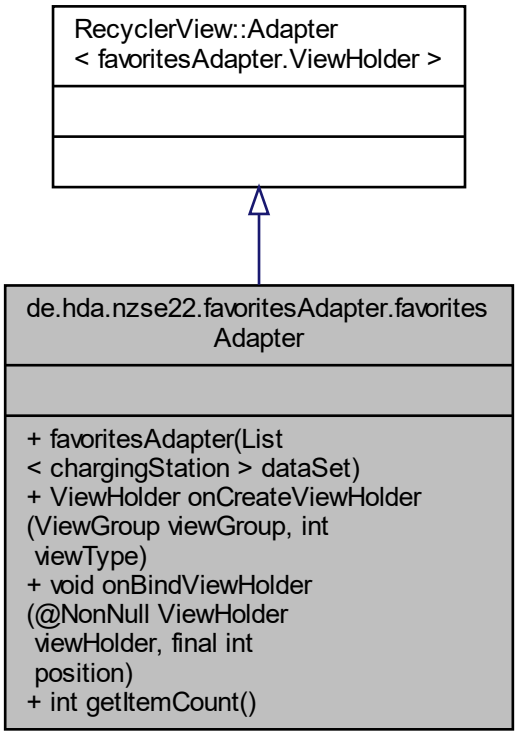
Definition at line 125 of file FavoriteActivity.java.

6.4 de.hda.nzse22.favoritesAdapter.favoritesAdapter Class Reference

Inheritance diagram for de.hda.nzse22.favoritesAdapter.favoritesAdapter:



Collaboration diagram for de.hda.nzse22.favoritesAdapter.favoritesAdapter:



Classes

- class **ViewHolder**

Public Member Functions

- favoritesAdapter (List< chargingStation > dataSet)
- ViewHolder onCreateViewHolder (ViewGroup viewGroup, int viewType)
- void onBindViewHolder (@NonNull ViewHolder viewHolder, final int position)
- int getItemCount ()

6.4.1 Detailed Description

Definition at line 27 of file favoritesAdapter.java.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 favoritesAdapter()

`de.hda.nzse22.favoritesAdapter.favoritesAdapter.favoritesAdapter ( List< chargingStation > dataSet )`

Initialize the dataset of the Adapter.

Parameters

dataSet	String[] containing the data to populate views to be used by RecyclerView.

Definition at line 40 of file favoritesAdapter.java.

6.4.3 Member Function Documentation

6.4.3.1 getItemCount() `int de.hda.nzse22.favoritesAdapter.favoritesAdapter.getItemCount ( )`

Returns

size of your dataset (invoked by the layout manager)

Definition at line 139 of file favoritesAdapter.java.

6.4.3.2 onBindViewHolder() `void de.hda.nzse22.favoritesAdapter.favoritesAdapter.onBindView↩Holder (`

```
@NonNull ViewHolder viewHolder,
final int position )
```

Initializes the Elements of the ViewHolder of the given Position

Parameters

viewHolder	Represents the content of the item
position	Position of the item in the adapter data set

Definition at line 71 of file favoritesAdapter.java.

6.4.3.3 onCreateViewHolder() `ViewHolder de.hda.nzse22.favoritesAdapter.favoritesAdapter.on↩CreateViewHolder (`

```
ViewGroup viewGroup,
int viewType )
```

Create new views (invoked by the layout manager)

Parameters

viewGroup	Group in which new View is added
viewType	type of the new View

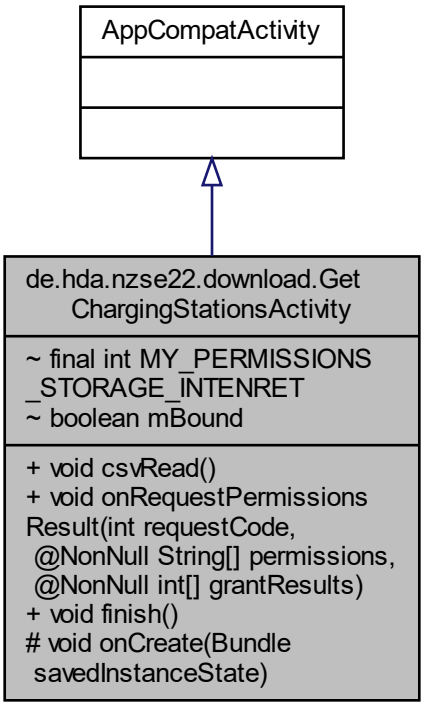
Returns

ViewHolder of a View of given viewType

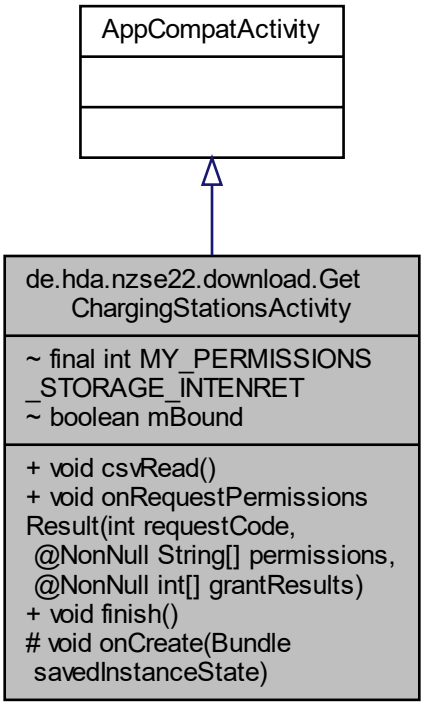
Definition at line 54 of file favoritesAdapter.java.

6.5 de.hda.nzse22.download.GetChargingStationsActivity Class Reference

Inheritance diagram for de.hda.nzse22.download.GetChargingStationsActivity:



Collaboration diagram for de.hda.nzse22.download.GetChargingStationsActivity:



Public Member Functions

- void csvRead () throws InterruptedException
- void onRequestPermissionsResult (int requestCode, @NonNull String[ ] permissions, @NonNull int[ ] grant↩ Results)
- void finish ()

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

6.5.1 Detailed Description

Definition at line 42 of file GetChargingStationsActivity.java.

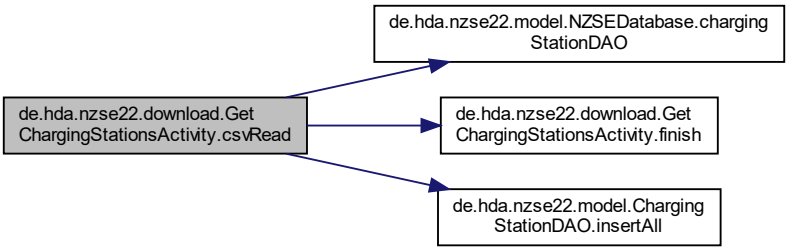
6.5.2 Member Function Documentation

6.5.2.1 csvRead() `void de.hda.nzse22.download.GetChargingStationsActivity.csvRead ( ) throws InterruptedException`

Reads downloaded csv file and cleans each line and creates new chargingStation objects and inserts them into the database

Definition at line 59 of file GetChargingStationsActivity.java.

Here is the call graph for this function:

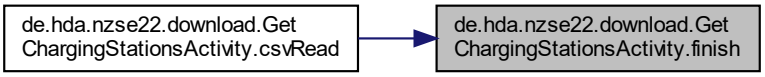


6.5.2.2 finish() `void de.hda.nzse22.download.GetChargingStationsActivity.finish ( )`

If activity finshed unbind service

Definition at line 200 of file GetChargingStationsActivity.java.

Here is the caller graph for this function:



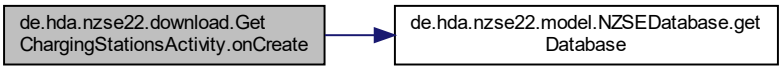
6.5.2.3 onCreate() `void de.hda.nzse22.download.GetChargingStationsActivity.onCreate ( Bundle savedInstanceState ) [protected]`

Parameters

savedInstanceState	-

Definition at line 134 of file GetChargingStationsActivity.java.

Here is the call graph for this function:



6.5.2.4 onRequestPermissionsResult()

```
void de.hda.nzse22.download.GetChargingStationsActivity.↩
onRequestPermissionsResult (
            int requestCode,
            @NonNull String[] permissions,
            @NonNull int[] grantResults )
```

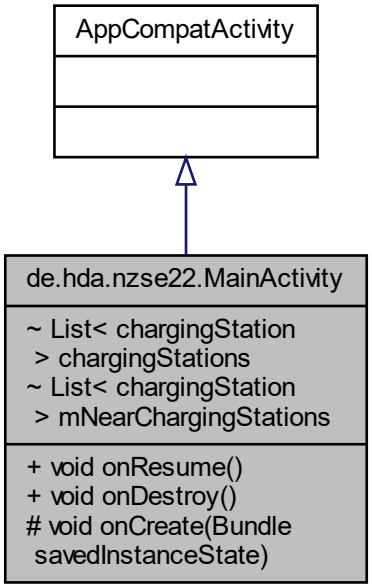
Parameters

requestCode	RequestCode of Permission
permissions	String array of permissions
grantResults	Granted permissions

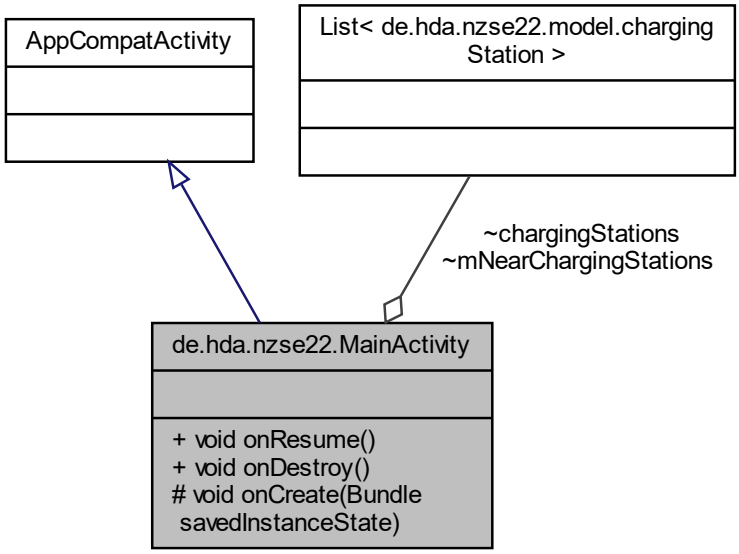
Definition at line 156 of file GetChargingStationsActivity.java.

6.6 de.hda.nzse22.MainActivity Class Reference

Inheritance diagram for de.hda.nzse22.MainActivity:



Collaboration diagram for de.hda.nzse22.MainActivity:



Public Member Functions

- void onResume ()
- void onDestroy ()

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

6.6.1 Detailed Description

Definition at line 35 of file MainActivity.java.

6.6.2 Member Function Documentation

6.6.2.1 onCreate() `void de.hda.nzse22.MainActivity.onCreate ( Bundle savedInstanceState ) [protected]`

Creates the activity and creates the database If it is empty it downloads a list of chargingstations in Germany If it is too big it deletes the database and downloads a list of chargingstations in Germany

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState(Bundle). (https://developer.android.com/reference/android/app/Activity#onCreate(android.os.Bundle))

Definition at line 61 of file MainActivity.java.

6.6.2.2 onDestroy() `void de.hda.nzse22.MainActivity.onDestroy ( )`

When activity is destroyed close database

Definition at line 302 of file MainActivity.java.

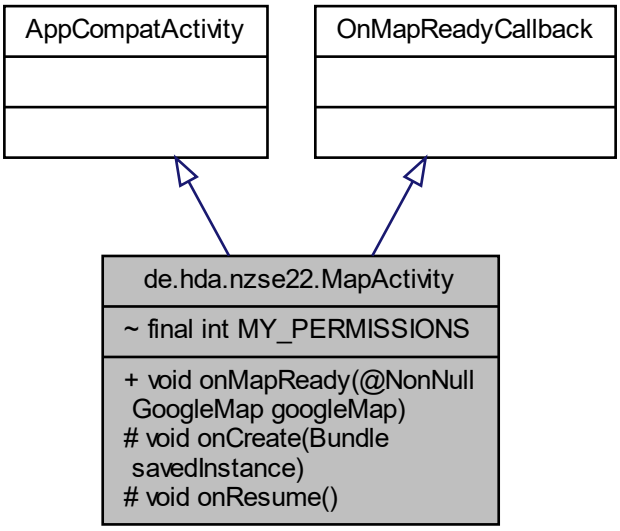
6.6.2.3 onResume() `void de.hda.nzse22.MainActivity.onResume ( )`

When activity is resumed reload favorite chargingstations

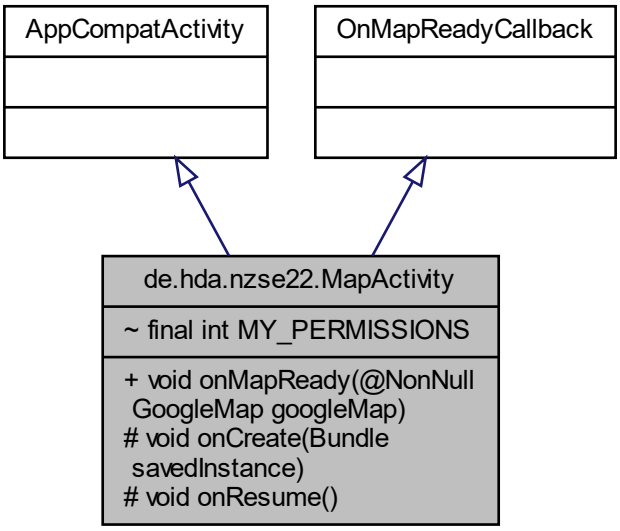
Definition at line 285 of file MainActivity.java.

6.7 de.hda.nzse22.MapActivity Class Reference

Inheritance diagram for de.hda.nzse22.MapActivity:



Collaboration diagram for de.hda.nzse22.MapActivity:



Public Member Functions

- void onMapReady (@NonNull GoogleMap googleMap)

Protected Member Functions

- void onCreate (Bundle savedInstance)
- void onResume ()

6.7.1 Detailed Description

Definition at line 35 of file MapActivity.java.

6.7.2 Member Function Documentation

6.7.2.1 onCreate() `void de.hda.nzse22.MapActivity.onCreate ( Bundle savedInstance ) [protected]`

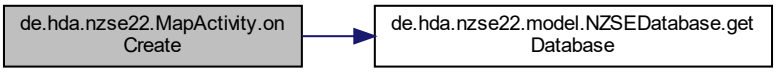
Creates the activity

Parameters

savedInstance	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState(Bundle). (https://developer.android.com/reference/android/app/Activity#onCreate(android.os.Bundle))

Definition at line 56 of file MapActivity.java.

Here is the call graph for this function:



6.7.2.2 onMapReady() `void de.hda.nzse22.MapActivity.onMapReady ( @NonNull GoogleMap googleMap )`

Initializes the map, marks the current location on the map and all chargingstations

Parameters

googleMap	GoogleMap for use with method

Definition at line 108 of file MapActivity.java.

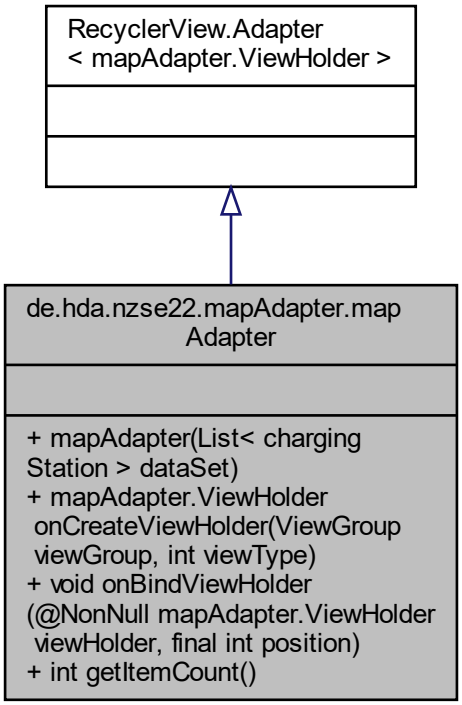
6.7.2.3 onResume() `void de.hda.nzse22.MapActivity.onResume ( ) [protected]`

Called if activity is resumed

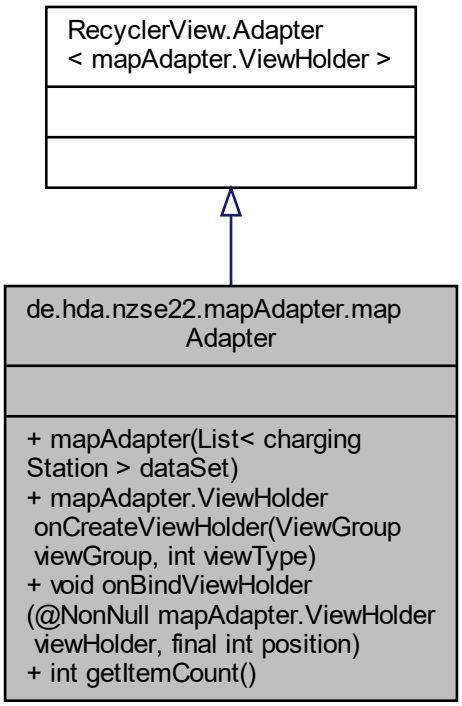
Definition at line 211 of file MapActivity.java.

6.8 de.hda.nzse22.mapAdapter.mapAdapter Class Reference

Inheritance diagram for de.hda.nzse22.mapAdapter.mapAdapter:



Collaboration diagram for de.hda.nzse22.mapAdapter.mapAdapter:



Classes

- class **ViewHolder**

Public Member Functions

- mapAdapter (List< chargingStation > dataSet)
- mapAdapter.ViewHolder onCreateViewHolder (ViewGroup viewGroup, int viewType)
- void onBindViewHolder (@NonNull mapAdapter.ViewHolder viewHolder, final int position)
- int getItemCount ()

6.8.1 Detailed Description

Definition at line 24 of file mapAdapter.java.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 mapAdapter() `de.hda.nzse22.mapAdapter.mapAdapter.mapAdapter ( List< chargingStation > dataSet )`

Initialize the dataset of the Adapter.

Parameters

dataSet	String[] containing the data to populate views to be used by RecyclerView.

Definition at line 36 of file mapAdapter.java.

6.8.3 Member Function Documentation

6.8.3.1 getItemCount() `int de.hda.nzse22.mapAdapter.mapAdapter.getItemCount ( )`

Returns

size of your dataset (invoked by the layout manager)

Definition at line 110 of file mapAdapter.java.

6.8.3.2 onBindViewHolder()

```
void de.hda.nzse22.mapAdapter.mapAdapter.onBindViewHolder (
        @NonNull mapAdapter.ViewHolder viewHolder,
        final int position )
```

Initializes the Elements of the ViewHolder of the given Position

Parameters

viewHolder	Represents the content of the item
position	Position of the item in the adapter data set

Definition at line 66 of file mapAdapter.java.

6.8.3.3 onCreateViewHolder()

```
mapAdapter.ViewHolder de.hda.nzse22.mapAdapter.mapAdapter.on↩
CreateViewHolder (
        ViewGroup viewGroup,
        int viewType )
```

Create new views (invoked by the layout manager)

Parameters

viewGroup	Group in which new View is added
viewType	type of the new View

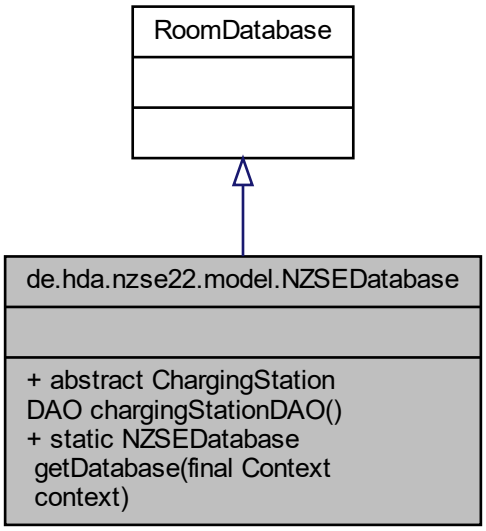
Returns

ViewHolder of a View of given viewType

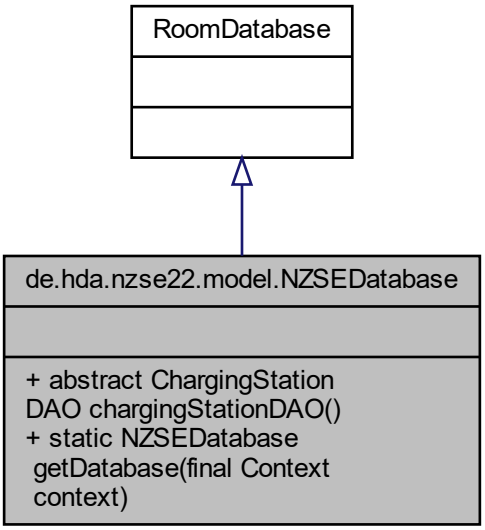
Definition at line 50 of file mapAdapter.java.

6.9 de.hda.nzse22.model.NZSEDatabase Class Reference

Inheritance diagram for de.hda.nzse22.model.NZSEDatabase:



Collaboration diagram for de.hda.nzse22.model.NZSEDatabase:



Public Member Functions

- abstract ChargingStationDAO chargingStationDAO ()

Static Public Member Functions

- static NZSEDatabase getDatabase (final Context context)

6.9.1 Detailed Description

Database for the chargingstations

Definition at line 13 of file NZSEDatabase.java.

6.9.2 Member Function Documentation

6.9.2.1 chargingStationDAO() `abstract ChargingStationDAO de.hda.nzse22.model.NZSEDatabase.↩ chargingStationDAO ( ) [abstract]`

chargingStation Data Access Object

Returns

chargingStationDAO

Here is the caller graph for this function:

6.9.2.2 getDatabase() `static NZSEDatabase de.hda.nzse22.model.NZSEDatabase.getDatabase ( final Context context ) [static]`

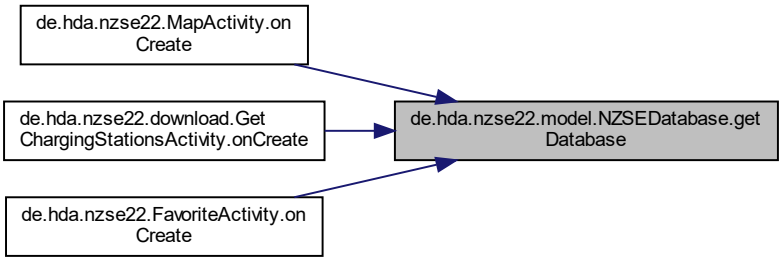
Returns the instance of the databases

Returns

Instance of the database

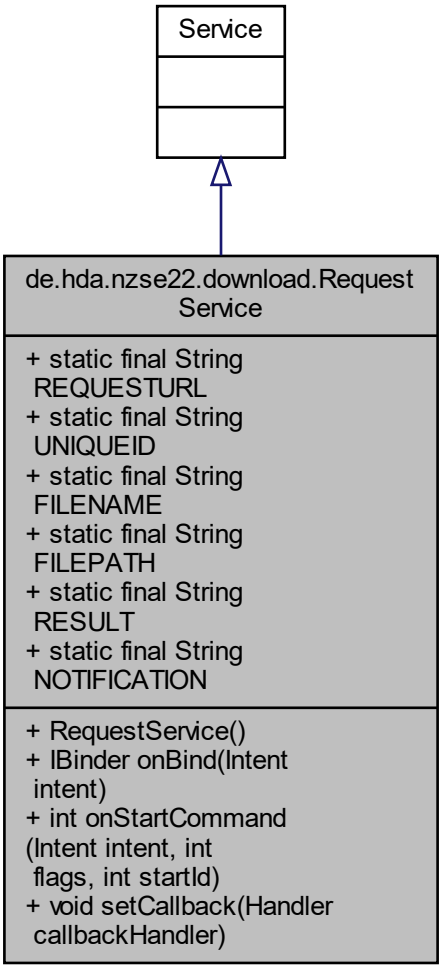
Definition at line 21 of file NZSEDatabase.java.

Here is the caller graph for this function:

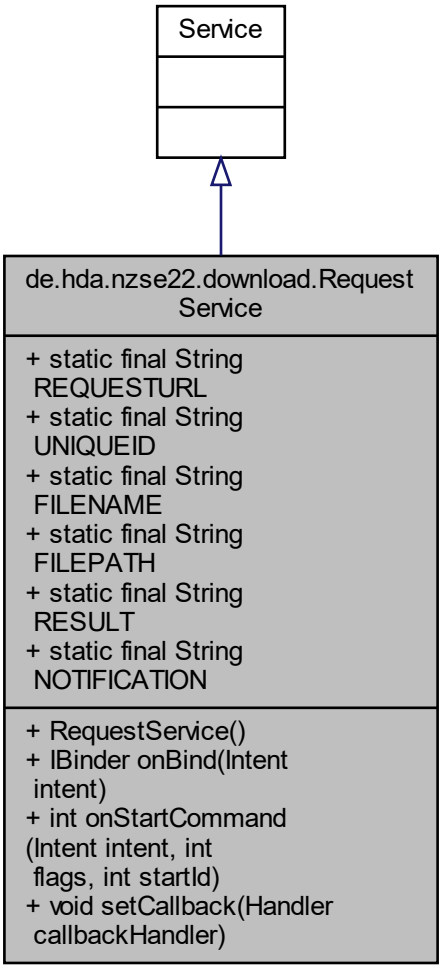


6.10 de.hda.nzse22.download.RequestService Class Reference

Inheritance diagram for de.hda.nzse22.download.RequestService:



Collaboration diagram for de.hda.nzse22.download.RequestService:



Classes

- class **DownloadThread**
- class RequestServiceBinder

Public Member Functions

- RequestService ()
- IBinder onBind (Intent intent)
- int onStartCommand (Intent intent, int flags, int startId)
- void setCallback (Handler callbackHandler)

Static Public Attributes

- static final String REQUESTURL = "NZSE.url"
- static final String UNIQUEID = "NZSE.FileId"
- static final String FILENAME = "NZSE.filename"
- static final String FILEPATH = "NZSE.filepath"
- static final String RESULT = "NZSE.result"
- static final String NOTIFICATION = "NZSE.notification"

6.10.1 Detailed Description

Created by NZSE Serverzugriff (Dateidownload) als Beispiel für eine Serviceklasse

Definition at line 25 of file RequestService.java.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 RequestService() `de.hda.nzse22.download.RequestService.RequestService ( )`

Konstruktor

Definition at line 61 of file RequestService.java.

6.10.3 Member Function Documentation

6.10.3.1 onBind() `IBinder de.hda.nzse22.download.RequestService.onBind ( Intent intent )`

Parameters

intent	- Intent, das beim Aufruf des Service verwendet wurde

Returns

binder - Kommunikationsobjekt zum Service

Definition at line 71 of file RequestService.java.

6.10.3.2 onStartCommand()

```
int de.hda.nzse22.download.RequestService.onStartCommand (
    Intent intent,
    int flags,
    int startId )
```

Methode wird vom System/android aufgerufen, wenn ein Service(mit startService) gestartet wird

Parameters

intent	- intent von startService Aufruf
flags	- flags ist 0 oder eine Kombination aus START_FLAG_REDELIVERY oder START_FLAG_RETRY
start↩ Id	- eindeutiger Id

Returns

Rückgabe ist einer der Werte START_STICKY_COMPATIBILITY, START_STICKY, START_NOT_STICKY oder START_REDELIVER_INTENT

Definition at line 87 of file RequestService.java.

6.10.3.3 setCallback() `void de.hda.nzse22.download.RequestService.setCallback ( Handler callbackHandler )`

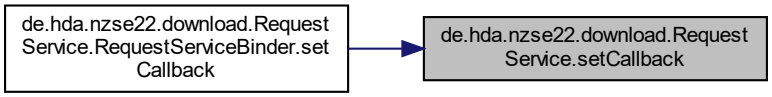
Callback registrieren

Parameters

callbackHandler	- Handler im Client bereitstellen

Definition at line 96 of file RequestService.java.

Here is the caller graph for this function:



6.10.4 Member Data Documentation

6.10.4.1 FILENAME `final String de.hda.nzse22.download.RequestService.FILENAME = "NZSE.↩ filename" [static]`

Bezeichner für Bundleparameter: Dateiname

Definition at line 37 of file RequestService.java.

6.10.4.2 FILEPATH `final String de.hda.nzse22.download.RequestService.FILEPATH = "NZSE.↩ filepath" [static]`

Bezeichner für Bundleparameter: Verzeichnispfad

Definition at line 41 of file RequestService.java.

6.10.4.3 NOTIFICATION `final String de.hda.nzse22.download.RequestService.NOTIFICATION = "NZSE.↩ notification" [static]`

Bezeichner für Bundleparameter: kurze Benachrichtigung

Definition at line 50 of file RequestService.java.

6.10.4.4 REQUESTURL `final String de.hda.nzse22.download.RequestService.REQUESTURL = "NZSE.↩ url" [static]`

Bezeichner für Bundleparameter: URL

Definition at line 29 of file RequestService.java.

6.10.4.5 RESULT `final String de.hda.nzse22.download.RequestService.RESULT = "NZSE.result" [static]`

Bezeichner für Bundleparameter: Activity.RESULT_OK oder Activity.RESULT_CANCEL

Definition at line 46 of file RequestService.java.

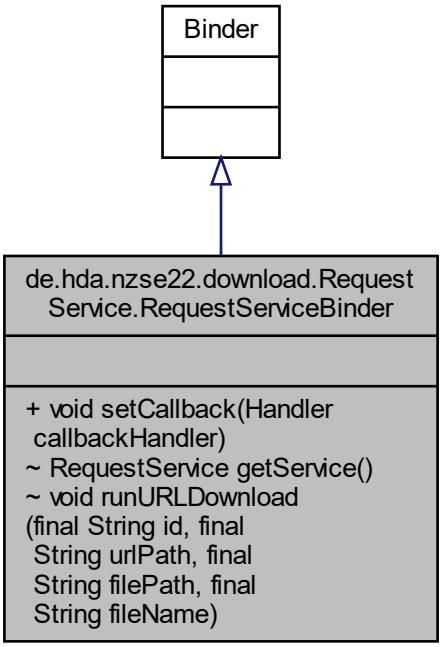
6.10.4.6 UNIQUEID `final String de.hda.nzse22.download.RequestService.UNIQUEID = "NZSE.FileId" [static]`

Bezeichner für Bundleparameter: Id zu Image

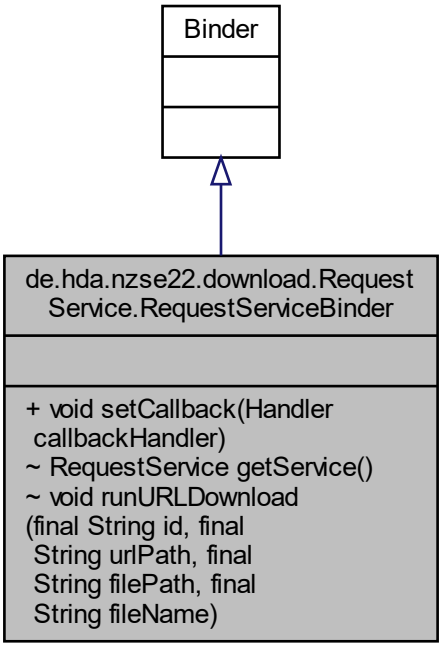
Definition at line 33 of file RequestService.java.

6.11 de.hda.nzse22.download.RequestService.RequestServiceBinder Class Reference

Inheritance diagram for de.hda.nzse22.download.RequestService.RequestServiceBinder:



Collaboration diagram for de.hda.nzse22.download.RequestService.RequestServiceBinder:



Public Member Functions

- void setCallback (Handler callbackHandler)

6.11.1 Detailed Description

Klasse RequestServiceBinder für den Client. Dieser Service läuft im selben Process ab wie die Client Activity; daher wird kein weiterer Kommunikationsaufwand (IPC) benötigt

Definition at line 139 of file RequestService.java.

6.11.2 Member Function Documentation

6.11.2.1 setCallback()

```
void de.hda.nzse22.download.RequestService.RequestServiceBinder.set↩
Callback (
            Handler callbackHandler )
```

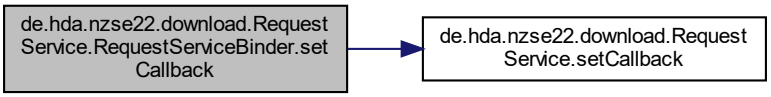
Für die Rückgabe von Ergebnissen

Parameters

callbackHandler	-

Definition at line 164 of file RequestService.java.

Here is the call graph for this function:



7 File Documentation

7.1 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/GetChargingStations↩ Activity.java File Reference

Classes

- class de.hda.nzse22.download.GetChargingStationsActivity

Packages

- package de.hda.nzse22.download

7.2 GetChargingStationsActivity.java

Go to the documentation of this file.

```
00001 package de.hda.nzse22.download;
00002
00003 import android.Manifest;
00004 import android.content.ComponentName;
00005 import android.content.Context;
00006 import android.content.Intent;
00007 import android.content.ServiceConnection;
00008 import android.content.pm.PackageManager;
00009 import android.os.Bundle;
00010 import android.os.Handler;
00011 import android.os.IBinder;
00012 import android.os.Looper;
00013 import android.os.Message;
00014 import android.widget.Toast;
00015
00016 import androidx.annotation.NonNull;
00017 import androidx.appcompat.app.AppCompatActivity;
00018 import androidx.core.app.ActivityCompat;
00019
00020 import com.opencsv.CSVParser;
00021 import com.opencsv.CSVParserBuilder;
00022 import com.opencsv.CSVReader;
00023 import com.opencsv.CSVReaderBuilder;
00024 import com.opencsv.exceptions.CsvValidationException;
00025
```

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.UnsupportedEncodingException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import de.hda.nzse22.R;
import de.hda.nzse22.model.ChargingStationDAO;
import de.hda.nzse22.model.NZSEDatabase;
import de.hda.nzse22.model.chargingStation;

public class GetChargingStationsActivity extends AppCompatActivity {

    final int MY_PERMISSIONS_STORAGE_INTENRET = 1;
    private final String filePath = "NZSE";
    private final String ladestationen = "ladestationen.txt"; // Name der lokalen Datei
    private final String url =
"https://www.bundesnetzagentur.de/SharedDocs/Downloads/DE/Sachgebiete/Energie/Unternehmen_Institutionen/E_Mobilitaet/Lad
    boolean mBound = false;
    private NZSEDatabase database;
    private Intent returnIntent;
    private String csvFile = "Empty"; // liefert die Downloadfunktion
    private RequestService mService = null;
    private RequestService.RequestServiceBinder binder;

    public void csvRead() throws InterruptedException {
        ChargingStationDAO chargingStationDAO = database.chargingStationDAO();
        List<chargingStation> allChargingStations = new ArrayList<>();
        int rows = 0;
        try {
            String[] csvline; // eingelesene csvZeile
            // lokale Datei ansprechen
            File myFile = new File(csvFile);
            FileInputStream fIn = new FileInputStream(myFile);
            InputStreamReader isr = new InputStreamReader(fIn, "ISO_8859-1");
            BufferedReader myReader = new BufferedReader(isr);

            CSVParser parser = new
CSVParserBuilder().withSeparator(';').withIgnoreQuotations(false).build();
            CSVReader csvReader = new
CSVReaderBuilder(myReader).withSkipLines(11).withCSVParser(parser).build();

            System.out.println("CSV-Datei einlesen startet ");

            while ((csvline = csvReader.readNext()) != null) { // Dateiende?
                allChargingStations.add(new chargingStation(Arrays.asList(csvline)));
                rows++;
            }
            csvReader.close();
            myReader.close();

            System.out.println("Chargingstations:  " + rows);
            Thread databaseInsertThread = new Thread(() ->
chargingStationDAO.insertAll(allChargingStations));
            databaseInsertThread.start();
            databaseInsertThread.join();
        } catch (CsvValidationException csvValidationException) {
            csvValidationException.printStackTrace();
        } catch (FileNotFoundException fileNotFoundException) {
            fileNotFoundException.printStackTrace();
        } catch (UnsupportedEncodingException unsupportedEncodingException) {
            unsupportedEncodingException.printStackTrace();
        } catch (IOException ioException) {
            ioException.printStackTrace();
        } catch (Exception e) {
            System.out.println(e.getMessage());
            e.printStackTrace();
        } finally {
            finish();
        }
        // try-catch
    } // csvRead

    private Handler getHandler() {
        return new Handler(Looper.getMainLooper()) {
            public void handleMessage(Message msg) {
                Bundle bundle = msg.getData();
                csvFile = (String) bundle.get(RequestService.FILEPATH);
                // Datei einlesen

                String uniqueId = (String) bundle.get(RequestService.UNIQUEID);
```

```
                String note = (String) bundle.get(RequestService.NOTIFICATION);
                Toast.makeText(GetChargingStationsActivity.this, uniqueId + " file:  " +
                        csvFile + " Bytes:  " + note, Toast.LENGTH_LONG).show();
                try {
                    csvRead();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }// handleMessage
        };
    }    // Serviceverbindung einrichten

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setTitle("Ladestationen");
        setContentView(R.layout.activity_download);
        database = NZSEDatabase.getDatabase(getApplicationContext());

        // Permission grant/gewähren
        String[] permissions = {Manifest.permission.INTERNET, Manifest.permission.WRITE_EXTERNAL_STORAGE};
        ActivityCompat.requestPermissions(this, permissions, MY_PERMISSIONS_STORAGE_INTENRET);

        // Bind to RequestService
        Intent myIntent = new Intent(this, RequestService.class);
        bindService(myIntent, mConnection, Context.BIND_AUTO_CREATE);

    }

    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == MY_PERMISSIONS_STORAGE_INTENRET) {// wenn die Anfrage gecancelled wird, sind die Ergebnisfelder leer.
            if (grantResults.length > 0) {
                for (int grant :  grantResults) {
                    if (grant == PackageManager.PERMISSION_GRANTED)
                        // Permissions wurden gewährt
                        System.out.println(permissions[grant] + " vorhanden");
                        // Berechtigungen stehen zu Verfügung, Zugriffe ausführen ..
                    else
                        System.out.println(permissions[grant] + "  n i c h t  vorhanden");
                    // Permissions werden abgelehnt, spezifische Zugriffe werden nicht ausgeführt
                }
            }
        }
    }
    private final ServiceConnection mConnection = new ServiceConnection() {

        @Override
        public void onServiceConnected(ComponentName className, IBinder service) {
            // an den RequestService binden,
            // Service-Objekt casting auf IBinder und LocalService instance erhalten
            binder = (RequestService.RequestServiceBinder) service;
            System.out.println(binder);
            mService = binder.getService();
            // callback setzen
            mService.setCallback(getHandler());
            System.out.println("DOWNLOAD starten");
            binder.runURLDownload("download", url, filePath, ladestationen);
            mBound = true;
        }

        @Override
        public void onServiceDisconnected(ComponentName arg0) {
            mBound = false;
        }
    };

    @Override
    public void finish() {
        System.out.println("******  bye bye");
        unbindService(mConnection);
        super.finish();
    }



} // GetChargingStationActivity
```

7.3 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/RequestService.java File Reference

Classes

- class de.hda.nzse22.download.RequestService
- class de.hda.nzse22.download.RequestService.RequestServiceBinder
- class **de.hda.nzse22.download.RequestService.DownloadThread**

Packages

- package de.hda.nzse22.download

7.4 RequestService.java

Go to the documentation of this file.

```
00001 package de.hda.nzse22.download;
00002
00003 import android.app.Service;
00004 import android.content.Intent;
00005 import android.os.Binder;
00006 import android.os.Bundle;
00007 import android.os.Handler;
00008 import android.os.IBinder;
00009 import android.os.Message;
00010 import android.util.Log;
00011
00012 import androidx.appcompat.app.AppCompatActivity;
00013
00014 import java.io.File;
00015 import java.io.FileOutputStream;
00016 import java.io.IOException;
00017 import java.io.InputStream;
00018 import java.net.HttpURLConnection;
00019 import java.net.URL;
00020
00025 public class RequestService extends Service {
00029     public static final String REQUESTURL = "NZSE.url";
00033     public static final String UNIQUEID = "NZSE.FileId";
00037     public static final String FILENAME = "NZSE.filename";
00041     public static final String FILEPATH = "NZSE.filepath";
00046     public static final String RESULT = "NZSE.result";
00050     public static final String NOTIFICATION = "NZSE.notification";
00054     private final IBinder mBinder = new RequestServiceBinder();
00055     private Handler uiServiceCallbackHandler;
00056     //----------------------------
00057
00061     public RequestService() {
00062         super();
00063         Log.i("RequestService", "*** wird erzeugt!  ***");
00064     }
00065
00070     @Override
00071     public IBinder onBind(Intent intent) {
00072         return mBinder;
00073     }
00074
00086     @Override
00087     public int onStartCommand(Intent intent, int flags, int startId) {
00088         return Service.START_STICKY;
00089     }
00090
00096     public void setCallback(Handler callbackHandler) {
00097         uiServiceCallbackHandler = callbackHandler;
00098     }
00099
00108     private void runURLDownload(final String id, final String urlPath, final String filePath, final
      String fileName) {
00109         DownloadThread dThread = new DownloadThread(id, urlPath, filePath, fileName);
00110         dThread.start();
00111     }
00112
00120     private void deliverResults(String uniqueId, String outputPath, int result, int size) {
00121
```

```
        Message msg = new Message();

        Bundle bundle = new Bundle();
        bundle.putString(NOTIFICATION, String.valueOf(size)); // oder "Download erfolgreich beendet.");
        bundle.putString(FILEPATH, outputPath);
        bundle.putString(UNIQUEID, uniqueId);
        bundle.putInt(RESULT, result);

        msg.setData(bundle);
        uiServiceCallbackHandler.sendMessage(msg); // Callback
    }

    public class RequestServiceBinder extends Binder {
        RequestService getService() {
            return RequestService.this;
        }

        void runURLDownload(final String id, final String urlPath, final String filePath, final String fileName) {
            getService().runURLDownload(id, urlPath, filePath, fileName);
        }

        public void setCallback(Handler callbackHandler) {
            getService().setCallback(callbackHandler);
        }

    } // RequestServiceBinder

    class DownloadThread extends Thread {
        private final String uniqueId;
        private final String urlPath;
        private final String filePath;
        private final String fileName;
        private int result;

        DownloadThread(final String id, final String urlPath, final String filePath, final String fileName) {
            this.result = AppCompatActivity.RESULT_CANCELED;
            this.uniqueId = id;
            this.urlPath = urlPath;
            this.filePath = filePath;
            this.fileName = fileName;
        }

        public void run() {
            try {
                String myfolder = getApplicationContext().getExternalCacheDir().getPath()/*Environment.getExternalStorageDirectory().getPath()*/ + "/" + filePath;
                System.out.println(myfolder);
                File f;
                f = new File(myfolder);
                System.out.println("(run) Verzeichnis:  " + myfolder);
                // erzeuge Verzeichnis (Folder), falls erforderlich
                if (!f.exists()) {
                    boolean b = f.mkdir();
                    if (!b)
                        Log.i("(run) NZSE: ", "Verzeichnis " + filePath + " konnte nicht erstellt werden.");
                }
                // Kontrollausgabe (wird eigentlich nicht benötigt)
                String[] dateien = new File(myfolder).list();
                assert dateien != null;
                for (String d :  dateien)
                    System.out.println("Datei im Verzeichnis:" + d);

                FileOutputStream fos = null;

                File output = new File(getApplicationContext().getExternalCacheDir().getPath()
                        /*Environment.getExternalStorageDirectory().getPath()*/ + "/" + this.filePath + "/" + this.fileName);

                if (output.exists()) output.delete();

                HttpURLConnection urlConnection = null;
                InputStream inputStream = null;
                int downloadedSize = 0;
                try {
                    Log.i("(run) FILE READER", "!  ...  some input **********************");
                    Log.i("(run) FILE READER", "!  ...  " + output.getPath());
                    System.out.println("(run) urlPath:  " + urlPath);
                    java.net.URL url = new URL(urlPath);
                    urlConnection = (HttpURLConnection) url.openConnection();
                    urlConnection.setRequestMethod("GET");

                    urlConnection.connect();
```

```java

                    //-----------------------------------------------
                    int responseCode = urlConnection.getResponseCode();
                    Log.i("**** **** FILE responseCode:", "...  " + responseCode);

                    System.out.println("...  einlesen:");
                    inputStream = urlConnection.getInputStream();

                    fos = new FileOutputStream(output.getPath());

                    byte[] buffer = new byte[2048];
                    int bufferLength;
                    while ((bufferLength = inputStream.read(buffer)) > 0) {
                        fos.write(buffer, 0, bufferLength);
                        downloadedSize += bufferLength;
                    }
                    Log.i("(run) Progress:  ", "downloadedSize:  " + downloadedSize);
                    // successfully finished
                } catch (Exception e) {
                    Log.i("(run) Exception:", " URLConnection");
                    e.printStackTrace();
                } finally {
                    if (urlConnection != null)
                        urlConnection.disconnect();
                    if (inputStream != null) {
                        try {
                            inputStream.close();
                        } catch (IOException e) {
                            Log.i("(run) Exception:", " InputStream null");
                            e.printStackTrace();
                        }
                    } // stream
                    if (fos != null) {
                        try {
                            fos.flush();
                            fos.close();
                        } catch (IOException e) {
                            Log.i("(run) Exception (1):", " close fails");
                            e.printStackTrace();
                        }
                    } // fos
                }// finally

                this.result = AppCompatActivity.RESULT_OK;
                deliverResults(this.uniqueId, output.getAbsolutePath(), this.result, downloadedSize);
            } catch (Exception e) {
                Log.i("(run) Exception(2):", e.getMessage());
            }
        }// run
    } // DownloadThread
} // RequestService
```

7.5 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/FavoriteActivity.java File Reference

Classes

- class de.hda.nzse22.FavoriteActivity

Packages

- package de.hda.nzse22

7.6 FavoriteActivity.java

Go to the documentation of this file.

```java
package de.hda.nzse22;

import android.Manifest;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.content.SharedPreferences;
import android.content.pm.PackageManager;
import android.location.Location;
```

```
00009 import android.location.LocationManager;
00010 import android.os.Bundle;
00011 import android.widget.SeekBar;
00012 import android.widget.TextView;
00013
00014 import androidx.annotation.NonNull;
00015 import androidx.appcompat.app.AppCompatActivity;
00016 import androidx.core.app.ActivityCompat;
00017 import androidx.recyclerview.widget.LinearLayoutManager;
00018 import androidx.recyclerview.widget.RecyclerView;
00019
00020 import com.google.android.gms.location.FusedLocationProviderClient;
00021 import com.google.android.gms.location.LocationServices;
00022 import com.google.android.gms.maps.model.LatLng;
00023 import com.google.android.material.bottomnavigation.BottomNavigationView;
00024
00025 import java.util.ArrayList;
00026 import java.util.List;
00027 import java.util.concurrent.atomic.AtomicReference;
00028
00029 import de.hda.nzse22.favoritesAdapter.favoritesAdapter;
00030 import de.hda.nzse22.model.NZSEDatabase;
00031 import de.hda.nzse22.model.chargingStation;
00032
00033 public class FavoriteActivity extends AppCompatActivity {
00034
00035     private final int MY_PERMISSIONS = 1;
00036     private final String[] permissions = {Manifest.permission.ACCESS_COARSE_LOCATION};
00037     private BottomNavigationView navView;
00038     private RecyclerView mRecyclerView;
00039     private favoritesAdapter mAdapter;
00040     private List<chargingStation> mFavoriteChargingStations = null;
00041     private NZSEDatabase database;
00042     private SeekBar distanceBar;
00043     private TextView distance;
00044     private int progressValue = 10;
00045     private SharedPreferences isServicetechniker;
00046
00047
00056     @Override
00057     protected void onCreate(Bundle savedInstanceState) {
00058         super.onCreate(savedInstanceState);
00059         setContentView(R.layout.activity_favorites);
00060         database = NZSEDatabase.getDatabase(getApplicationContext());
00061         initSharedPreferences();
00062         setList();
00063         initNavBar();
00064         initSeekBar();
00065     }
00066
00070     private void initSharedPreferences() {
00071         isServicetechniker = getSharedPreferences("NZSE_SS22", MODE_PRIVATE);
00072     }
00073
00079     void populateFavoriteList() throws InterruptedException {
00080         if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
00081                 != PackageManager.PERMISSION_GRANTED &&
00082                 ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION)
00083                         != PackageManager.PERMISSION_GRANTED) {
00084             ActivityCompat.requestPermissions(this, permissions, MY_PERMISSIONS);
00085         }
00086
00087         mFavoriteChargingStations = new ArrayList<>();
00088
00089         AtomicReference<List<chargingStation» dbList = new AtomicReference<>();
00090         Thread dbListThread;
00091         if (!isServicetechniker.getBoolean("isServicetechniker", false)) {
00092             dbListThread = new Thread(() -> dbList.set(database.chargingStationDAO().getFavorites()));
00093         } else {
00094             dbListThread = new Thread(() ->
      dbList.set(database.chargingStationDAO().getNotWorkingChargingStations()));
00095         }
00096         dbListThread.start();
00097         dbListThread.join();
00098
00099         FusedLocationProviderClient fusedLocationClient =
      LocationServices.getFusedLocationProviderClient(this);
00100         fusedLocationClient.getLastLocation().addOnSuccessListener(this, location -> {
00101             if (location != null) {
00102                 LatLng currentLocation = new LatLng(location.getLatitude(), location.getLongitude());
00103                 for (chargingStation favorite :  dbList.get()) {
00104                     LatLng favoriteCoordination = new LatLng(favorite.getLatitude(),
      favorite.getLongitude());
00105                     if (checkIsInDistance(favoriteCoordination, currentLocation, progressValue)) {
00106                         mFavoriteChargingStations.add(favorite);
00107                     }
00108                     mFavoriteChargingStations.sort((o1, o2) -> {
```

```
                        LatLng chargingStation1 = new LatLng(o1.getLatitude(), o1.getLongitude());
                        LatLng chargingStation2 = new LatLng(o2.getLatitude(), o2.getLongitude());
                        int distanceStation1 = checkDistance(chargingStation1, currentLocation);
                        int distanceStation2 = checkDistance(chargingStation2, currentLocation);
                        return Integer.compare(distanceStation1, distanceStation2);
                    });
                }
                initRecycler();
            }
        });
    }

    @Override
    protected void onResume() {
        initNavBar();
        setList();
        initRecycler();
        initSharedPreferences();
        super.onResume();
    }

    private void setList() {
        try {
            populateFavoriteList();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    private void initRecycler() {
        mAdapter = new favoritesAdapter(mFavoriteChargingStations);
        mRecyclerView = findViewById(R.id.favoritesRecycler);
        mRecyclerView.setAdapter(mAdapter);
        LinearLayoutManager linearLayoutManager = new LinearLayoutManager(getApplicationContext());
        mRecyclerView.setLayoutManager(linearLayoutManager);
    }

    @SuppressLint("NonConstantResourceId")
    private void initNavBar() {
        navView = findViewById(R.id.bottom_navigation_favorites);
        navView.setSelectedItemId(R.id.navigation_favorites);
        navView.getMenu().performIdentifierAction(R.id.navigation_favorites, 0);
        navView.setOnItemSelectedListener(item -> {
            switch (item.getItemId()) {
                case R.id.navigation_home:
                    startActivity(new Intent(getApplicationContext(), MainActivity.class));
                    overridePendingTransition(0, 0);
                    return true;
                case R.id.navigation_favorites:
                    return true;
                case R.id.navigation_map:
                    startActivity(new Intent(getApplicationContext(), MapActivity.class));
                    overridePendingTransition(0, 0);
                    return true;
            }
            return false;
        });
    }

    private void initSeekBar() {
        distance = findViewById(R.id.distanceFavoriteText);
        distanceBar = findViewById(R.id.seekBarFavorites);
        String distanceText = progressValue + " km";
        distance.setText(distanceText);
        distanceBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {


            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                if (fromUser) {
                    if (progress >= 0 && progress <= seekBar.getMax()) {
                        progressValue = progress / 10;
                        progressValue = progressValue * 10;
                        String distanceText = progressValue + " km";
                        distance.setText(distanceText);
                    }
                }
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {
            }

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {
                setList();
                initRecycler();
            }
```

```
        });
    }

    private boolean checkIsInDistance(LatLng favoriteCoordinates, LatLng lastCoordinates, int maxDistance) {
        return checkDistance(favoriteCoordinates, lastCoordinates) <= maxDistance;
    }

    private int checkDistance(@NonNull LatLng favoriteCoordinates, @NonNull LatLng lastCoordinates) {
        Location currentLocation = new Location(LocationManager.GPS_PROVIDER);
        currentLocation.setLatitude(lastCoordinates.latitude);
        currentLocation.setLongitude(lastCoordinates.longitude);

        Location favoriteLocation = new Location(LocationManager.GPS_PROVIDER);
        favoriteLocation.setLatitude(favoriteCoordinates.latitude);
        favoriteLocation.setLongitude(favoriteCoordinates.longitude);

        int distance = (int) currentLocation.distanceTo(favoriteLocation);
        distance /= 1000;
        return distance;
    }


}
```

7.7 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/favoritesAdapter/favoritesAdapter.java File Reference

Classes

- class de.hda.nzse22.favoritesAdapter.favoritesAdapter
- class **de.hda.nzse22.favoritesAdapter.favoritesAdapter.ViewHolder**

Packages

- package de.hda.nzse22.favoritesAdapter

7.8 favoritesAdapter.java

Go to the documentation of this file.

```
package de.hda.nzse22.favoritesAdapter;

import static android.content.Context.MODE_PRIVATE;

import android.content.Context;
import android.content.Intent;
import android.content.SharedPreferences;
import android.net.Uri;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.ToggleButton;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

import de.hda.nzse22.R;
import de.hda.nzse22.model.ChargingStationDAO;
import de.hda.nzse22.model.NZSEDatabase;
import de.hda.nzse22.model.chargingStation;

public class favoritesAdapter extends RecyclerView.Adapter<favoritesAdapter.ViewHolder> {

    private final List<chargingStation> localDataSet;
    private Context mContext;
    private SharedPreferences isWorking;
```

```java


    public favoritesAdapter(List<chargingStation> dataSet) {
        localDataSet = dataSet;
    }


    @NonNull
    @Override
    public ViewHolder onCreateViewHolder(ViewGroup viewGroup, int viewType) {
        // Create a new view, which defines the UI of the list item
        View view = LayoutInflater.from(viewGroup.getContext())
                .inflate(R.layout.recyclerview_favorites, viewGroup, false);
        mContext = viewGroup.getContext();
        isWorking = mContext.getSharedPreferences("NZSE_SS22", MODE_PRIVATE);
        return new ViewHolder(view);
    }


    @Override
    public void onBindViewHolder(@NonNull ViewHolder viewHolder, final int position) {
        viewHolder.operatorTextView.setText(localDataSet.get(position).getOperator());
        viewHolder.addressTextView.setText(localDataSet.get(position).getAddress());
        viewHolder.cityTextView.setText(localDataSet.get(position).getCity());

        viewHolder.plugType.setText(localDataSet.get(position).getPlugType1());

        if (isWorking.getBoolean("isServicetechniker", false)) {
            viewHolder.favoriteButton.setVisibility(View.INVISIBLE);
        } else {
            viewHolder.favoriteButton.setVisibility(View.VISIBLE);
            if (localDataSet.get(position).getFavorite()) {
                viewHolder.favoriteButton.setBackgroundResource(android.R.drawable.star_big_on);
            } else {
                viewHolder.favoriteButton.setBackgroundResource(android.R.drawable.star_big_off);
            }
        }
        if (localDataSet.get(position).isWorking()) {
            viewHolder.workingView.setImageResource(android.R.drawable.presence_online);
        } else {
            viewHolder.workingView.setImageResource(android.R.drawable.presence_busy);
        }

        if (localDataSet.get(position).isExpand()) {
            viewHolder.expandButton.setBackgroundResource(android.R.drawable.arrow_up_float);
            viewHolder.detailsView.setVisibility(View.VISIBLE);
        } else {
            viewHolder.expandButton.setBackgroundResource(android.R.drawable.arrow_down_float);
            viewHolder.detailsView.setVisibility(View.GONE);
        }

        if (isWorking.getBoolean("isServicetechniker", false)) {
            viewHolder.reportButton.setText(R.string.repariert);
        }

        viewHolder.favoriteButton.setOnClickListener(v -> {
            localDataSet.get(position).setFavorite(!localDataSet.get(position).getFavorite());
            updateChargingStation(position);
            notifyItemChanged(position);
        });

        viewHolder.expandButton.setOnClickListener(l -> {
            localDataSet.get(position).setExpand(!localDataSet.get(position).isExpand());
            updateChargingStation(position);
            notifyItemChanged(position);

        });

        viewHolder.reportButton.setOnClickListener(l -> {
            localDataSet.get(position).setWorking(isWorking.getBoolean("isServicetechniker", false));
            updateChargingStation(position);
            notifyItemChanged(position);
        });

        viewHolder.navigationButton.setOnClickListener(l -> {
            String geoLocation = "google.navigation:q=" + localDataSet.get(position).getLatitude() +
    "," +
                    localDataSet.get(position).getLongitude();
            Uri gmmIntentUri = Uri.parse(geoLocation);
            Intent navigationIntent = new Intent(Intent.ACTION_VIEW, gmmIntentUri);
            navigationIntent.setPackage("com.google.android.apps.maps");
            mContext.startActivity(navigationIntent);
        });
    }

    @Override
    public int getItemCount() {
```

```
        return localDataSet.size();
    }

    private void updateChargingStation(int position) {
        NZSEDatabase db = NZSEDatabase.getDatabase(mContext);
        ChargingStationDAO dao = db.chargingStationDAO();
        Thread updateThread = new Thread(() -> dao.update(localDataSet.get(position)));
        updateThread.start();
        try {
            updateThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }


    public static class ViewHolder extends RecyclerView.ViewHolder {
        private final TextView addressTextView;
        private final TextView operatorTextView;
        private final ToggleButton favoriteButton;
        private final ToggleButton expandButton;
        private final TextView cityTextView;
        private final ImageView workingView;
        private final TextView plugType;
        private final View detailsView;

        private final Button reportButton;
        private final Button navigationButton;


        public ViewHolder(View view) {
            super(view);
            operatorTextView = view.findViewById(R.id.operatorRecycler);
            addressTextView = view.findViewById(R.id.addressRecycler);
            favoriteButton = view.findViewById(R.id.favoriteRecycler);
            cityTextView = view.findViewById(R.id.cityRecycler);
            workingView = view.findViewById(R.id.workingView);
            expandButton = view.findViewById(R.id.expandButton);

            detailsView = view.findViewById(R.id.detailsLayout);
            plugType = view.findViewById(R.id.plugtypeText);
            reportButton = view.findViewById(R.id.reportButton);
            navigationButton = view.findViewById(R.id.routeButton);
        }

    }
}
```

7.9 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MainActivity.java File Reference

Classes

- class de.hda.nzse22.MainActivity

Packages

- package de.hda.nzse22

7.10 MainActivity.java

Go to the documentation of this file.

```
package de.hda.nzse22;


import android.Manifest;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.content.SharedPreferences;
import android.content.pm.PackageManager;
import android.location.Location;
import android.location.LocationManager;
import android.os.Bundle;

```

```
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.SwitchCompat;
import androidx.core.app.ActivityCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.material.bottomnavigation.BottomNavigationView;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicReference;

import de.hda.nzse22.download.GetChargingStationsActivity;
import de.hda.nzse22.favoritesAdapter.favoritesAdapter;
import de.hda.nzse22.model.ChargingStationDAO;
import de.hda.nzse22.model.NZSEDatabase;
import de.hda.nzse22.model.chargingStation;

public class MainActivity extends AppCompatActivity {

    private final int MY_PERMISSIONS = 1;
    private final String[] permissions = {Manifest.permission.ACCESS_COARSE_LOCATION, Manifest.permission.INTERNET, Manifest.permission.WRITE_EXTERNAL_STORAGE};
    List<chargingStation> chargingStations = null;
    List<chargingStation> mNearChargingStations = null;
    private NZSEDatabase database;
    private ChargingStationDAO chargingStationDAO;
    private BottomNavigationView navView;
    private favoritesAdapter mAdapter;
    private RecyclerView mRecyclerView;
    private SwitchCompat servicetechnikerSwitch;
    private SharedPreferences sharedPreferences;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        ActivityCompat.requestPermissions(this, permissions, MY_PERMISSIONS);
        createDatabase();
        initServiceTechnikerSwitch();
        setList();
        initNavView();
        Integer amountofChargingStations = 29612;

        if (getDatabaseSize() < amountofChargingStations) {
            startActivity(new Intent(getApplicationContext(), GetChargingStationsActivity.class));
        } else if (getDatabaseSize() > amountofChargingStations) {
            deleteDatabase();
            startActivity(new Intent(getApplicationContext(), GetChargingStationsActivity.class));
        }

    }

    private void initServiceTechnikerSwitch() {
        sharedPreferences = getSharedPreferences("NZSE_SS22", MODE_PRIVATE);
        servicetechnikerSwitch = findViewById(R.id.servicetechnikerSwitch);
        if (!sharedPreferences.contains("isServicetechniker")) {
            SharedPreferences.Editor checkServicetechniker = sharedPreferences.edit();
            checkServicetechniker.putBoolean("isServicetechniker", false);
            checkServicetechniker.apply();
        }
        servicetechnikerSwitch.setOnClickListener(l -> {
            SharedPreferences.Editor checkServicetechniker = sharedPreferences.edit();
            checkServicetechniker.putBoolean("isServicetechniker", servicetechnikerSwitch.isChecked());
            checkServicetechniker.apply();
            try {
                populateChargingStation();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    }

    @SuppressLint("NonConstantResourceId")
    private void initNavView() {
        navView = findViewById(R.id.bottom_navigation_main);

        navView.setSelectedItemId(R.id.navigation_home);
        navView.getMenu().performIdentifierAction(R.id.navigation_home, 0);

        navView.setOnItemSelectedListener(item -> {
```

```
            switch (item.getItemId()) {
                case R.id.navigation_home:
                    return true;
                case R.id.navigation_favorites:
                    startActivity(new Intent(getApplicationContext(), FavoriteActivity.class));
                    overridePendingTransition(0, 0);
                    return true;
                case R.id.navigation_map:
                    startActivity(new Intent(getApplicationContext(), MapActivity.class));
                    overridePendingTransition(0, 0);
                    return true;
            }
            return false;
        });
    }

    private void initRecycler() {
        mAdapter = new favoritesAdapter(mNearChargingStations);
        mRecyclerView = findViewById(R.id.recyclerViewMain);
        mRecyclerView.setAdapter(mAdapter);
        LinearLayoutManager linearLayoutManager = new LinearLayoutManager(getApplicationContext());
        mRecyclerView.setLayoutManager(linearLayoutManager);
    }

    private void setList() {
        try {
            populateChargingStation();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    private void populateStationList() throws InterruptedException {
        if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED &&
                ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION)
!= PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this, permissions, MY_PERMISSIONS);
        }

        mNearChargingStations = new ArrayList<>();

        if (sharedPreferences.getBoolean("isServicetechniker", false)) {
            AtomicReference<List<chargingStation» dbList = new AtomicReference<>();
            Thread dbListThread = new Thread(() ->
dbList.set(database.chargingStationDAO().getNotWorkingChargingStations()));
            ;
            dbListThread.start();
            dbListThread.join();
            chargingStations = dbList.get();
        }

        FusedLocationProviderClient fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this);
        fusedLocationClient.getLastLocation().addOnSuccessListener(this, location -> {
            if (location != null) {
                LatLng currentLocation = new LatLng(location.getLatitude(), location.getLongitude());
                for (chargingStation favorite :  chargingStations) {
                    LatLng favoriteCoordination = new LatLng(favorite.getLatitude(),
favorite.getLongitude());
                    if (checkIsInDistance(favoriteCoordination, currentLocation, 25)) {
                        mNearChargingStations.add(favorite);
                    }
                    mNearChargingStations.sort((o1, o2) -> {
                        LatLng chargingStation1 = new LatLng(o1.getLatitude(), o1.getLongitude());
                        LatLng chargingStation2 = new LatLng(o2.getLatitude(), o2.getLongitude());
                        int distanceStation1 = checkDistance(chargingStation1, currentLocation);
                        int distanceStation2 = checkDistance(chargingStation2, currentLocation);
                        return Integer.compare(distanceStation1, distanceStation2);
                    });
                }
                initRecycler();
            }

        });
    }

    private boolean checkIsInDistance(LatLng favoriteCoordinates, LatLng lastCoordinates, int
maxDistance) {
        return checkDistance(favoriteCoordinates, lastCoordinates) <= maxDistance;
    }

    private int checkDistance(@NonNull LatLng favoriteCoordinates, @NonNull LatLng lastCoordinates) {
        Location currentLocation = new Location(LocationManager.GPS_PROVIDER);
        currentLocation.setLatitude(lastCoordinates.latitude);
        currentLocation.setLongitude(lastCoordinates.longitude);

```

```
        Location favoriteLocation = new Location(LocationManager.GPS_PROVIDER);
        favoriteLocation.setLatitude(favoriteCoordinates.latitude);
        favoriteLocation.setLongitude(favoriteCoordinates.longitude);

        int distance = (int) currentLocation.distanceTo(favoriteLocation);
        distance /= 1000;
        return distance;
    }

    private void populateChargingStation() throws InterruptedException {
        Thread populateThread = new Thread(() -> chargingStations = chargingStationDAO.getAll());
        populateThread.start();
        populateThread.join();
        populateStationList();
    }

    private void createDatabase() {
        database = NZSEDatabase.getDatabase(getApplicationContext());
        chargingStationDAO = database.chargingStationDAO();
    }

    private Integer getDatabaseSize() {
        AtomicReference<Integer> sizeDB = new AtomicReference<>(0);
        Thread checkThread = new Thread(() -> {
            sizeDB.set(chargingStationDAO.getAll().size());
        });
        checkThread.start();
        try {
            checkThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        return sizeDB.get();
    }

    private void deleteDatabase() {
        Thread checkThread = new Thread(() -> {
            chargingStationDAO.deleteAll();
        });
        checkThread.start();
        try {
            checkThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public void onResume() {
        navView.setSelectedItemId(R.id.navigation_home);
        sharedPreferences = getSharedPreferences("NZSE_SS22", MODE_PRIVATE);
        super.onResume();
        try {
            populateChargingStation();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(chargingStations.size());
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        database.close();
    }
}
```

7.11 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MapActivity.java File Reference

Classes

- class de.hda.nzse22.MapActivity

Packages

- package de.hda.nzse22

7.12 MapActivity.java

Go to the documentation of this file.

```
package de.hda.nzse22;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.location.Location;
import android.location.LocationManager;
import android.os.Bundle;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.MarkerOptions;
import com.google.android.material.bottomnavigation.BottomNavigationView;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicReference;

import de.hda.nzse22.mapAdapter.mapAdapter;
import de.hda.nzse22.model.NZSEDatabase;
import de.hda.nzse22.model.chargingStation;


public class MapActivity extends AppCompatActivity implements OnMapReadyCallback {

    final int MY_PERMISSIONS = 1;
    private final String[] permissions = {Manifest.permission.ACCESS_COARSE_LOCATION};
    private GoogleMap mGoogleMap;
    private NZSEDatabase database;
    private List<chargingStation> chargingStationList = new ArrayList<>();
    private RecyclerView mRecyclerViewMap;
    private mapAdapter mAdapter;
    private FusedLocationProviderClient fusedLocationClient;
    private LatLng lastLocation;

    @Override
    protected void onCreate(Bundle savedInstance) {

        super.onCreate(savedInstance);
        setContentView(R.layout.activity_map);
        database = NZSEDatabase.getDatabase(getApplicationContext());
        try {
            chargingStationList = loadChargingStations();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(chargingStationList.size());
        fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);
        BottomNavigationView navView = findViewById(R.id.bottom_navigation_map);

        initNavView(navView);

        // Get the SupportMapFragment and request notification when the map is ready to be used.
        SupportMapFragment mapFragment = (SupportMapFragment) getSupportFragmentManager()
                .findFragmentById(R.id.map);
        assert mapFragment != null;
        mapFragment.getMapAsync(this);

    }

    private void initNavView(BottomNavigationView navView) {
        navView.setSelectedItemId(R.id.navigation_map);
        navView.getMenu().performIdentifierAction(R.id.navigation_map, 0);

        navView.setOnItemSelectedListener(item -> {
            if (item.getItemId() == R.id.navigation_home) {
                startActivity(new Intent(getApplicationContext(), MainActivity.class));
                overridePendingTransition(0, 0);
                return true;
            } else if (item.getItemId() == R.id.navigation_favorites) {
                startActivity(new Intent(getApplicationContext(), FavoriteActivity.class));
                overridePendingTransition(0, 0);
```

```
                return true;
            } else return item.getItemId() == R.id.navigation_map;
        });
    }

    @Override
    public void onMapReady(@NonNull GoogleMap googleMap) {
        mGoogleMap = googleMap;
        mGoogleMap.getUiSettings().setMyLocationButtonEnabled(true);
        if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED && ActivityCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_COARSE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this, permissions, MY_PERMISSIONS);
            return;
        }
        mGoogleMap.setMyLocationEnabled(true);

        fusedLocationClient.getLastLocation().addOnSuccessListener(this, location -> {
            if (location != null) {
                lastLocation = new LatLng(location.getLatitude(), location.getLongitude());
                mGoogleMap.addMarker(new MarkerOptions().position(lastLocation).title("Marker on Last
Location"));
                float zoom = 12.0f;
                mGoogleMap.moveCamera(CameraUpdateFactory.newLatLngZoom(lastLocation, zoom));
                addMarker();
            }
        });
    }

    private void addMarker() {
        for (chargingStation chargingStatonsObject :  chargingStationList) {
            Location chargingStationLocation = new Location(LocationManager.GPS_PROVIDER);
            chargingStationLocation.setLongitude(chargingStatonsObject.getLongitude());
            chargingStationLocation.setLatitude(chargingStatonsObject.getLatitude());
            if (checkIsInDistance(chargingStationLocation, lastLocation, 25)) {
                LatLng chargingStationCoordinates = new LatLng(chargingStatonsObject.getLatitude(),
chargingStatonsObject.getLongitude());
                mGoogleMap.addMarker(new
MarkerOptions().position(chargingStationCoordinates).title(chargingStatonsObject.getOperator()));
                mGoogleMap.setOnMarkerClickListener(marker -> {
                    mRecyclerViewMap = findViewById(R.id.recyclerViewMapMarker);
                    LinearLayoutManager linearLayoutManager = new
LinearLayoutManager(getApplicationContext());
                    mRecyclerViewMap.setLayoutManager(linearLayoutManager);
                    double longitude = marker.getPosition().longitude;
                    double latitude = marker.getPosition().latitude;

                    try {
                        mAdapter = new mapAdapter(loadChargingStationWithCoordinates(longitude,
latitude));
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    mRecyclerViewMap.setAdapter(mAdapter);
                    return false;
                });
            }

        }
    }

    private List<chargingStation> loadChargingStations() throws InterruptedException {
        AtomicReference<List<chargingStation» chargingStations = new AtomicReference<>();
        Thread dbListThread = new Thread(() ->
chargingStations.set(database.chargingStationDAO().getAll()));
        dbListThread.start();
        dbListThread.join();
        return chargingStations.get();
    }

    private List<chargingStation> loadChargingStationWithCoordinates(double longitude, double
latitude) throws InterruptedException {
        AtomicReference<List<chargingStation» chargingStations = new AtomicReference<>();
        Thread dbListThread = new Thread(() ->
chargingStations.set(database.chargingStationDAO().getChargingStationsWithCoordinates(latitude,
longitude)));
        dbListThread.start();
        dbListThread.join();
        return chargingStations.get();
    }

    private boolean checkIsInDistance(Location checkLocation, LatLng lastLocation, int maxDistance) {

        Location currentLocation = new Location(LocationManager.GPS_PROVIDER);
        currentLocation.setLatitude(lastLocation.latitude);
        currentLocation.setLongitude(lastLocation.longitude);
        int distance = (int) currentLocation.distanceTo(checkLocation);
```

```
        distance /= 1000;
        return distance <= maxDistance;
    }

    @Override
    protected void onResume() {

        super.onResume();
    }

}
```

7.13 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/mapAdapter/mapAdapter.java File Reference

Classes

- class de.hda.nzse22.mapAdapter.mapAdapter
- class **de.hda.nzse22.mapAdapter.mapAdapter.ViewHolder**

Packages

- package de.hda.nzse22.mapAdapter

7.14 mapAdapter.java

Go to the documentation of this file.

```
package de.hda.nzse22.mapAdapter;

import android.content.Context;
import android.content.Intent;
import android.net.Uri;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.ToggleButton;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

import de.hda.nzse22.R;
import de.hda.nzse22.model.ChargingStationDAO;
import de.hda.nzse22.model.NZSEDatabase;
import de.hda.nzse22.model.chargingStation;

public class mapAdapter extends RecyclerView.Adapter<mapAdapter.ViewHolder> {

    private final List<chargingStation> localDataSet;
    private Context mContext;


    public mapAdapter(List<chargingStation> dataSet) {
        localDataSet = dataSet;
    }


    @NonNull
    @Override
    public mapAdapter.ViewHolder onCreateViewHolder(ViewGroup viewGroup, int viewType) {
        // Create a new view, which defines the UI of the list item
        View view = LayoutInflater.from(viewGroup.getContext())
                .inflate(R.layout.recyclerview_map, viewGroup, false);
        mContext = viewGroup.getContext();
        return new mapAdapter.ViewHolder(view);
    }

```

```java

    @Override
    public void onBindViewHolder(@NonNull mapAdapter.ViewHolder viewHolder, final int position) {
        viewHolder.operatorTextView.setText(localDataSet.get(position).getOperator());
        viewHolder.addressTextView.setText(localDataSet.get(position).getAddress());
        viewHolder.cityTextView.setText(localDataSet.get(position).getCity());

        if (localDataSet.get(position).getFavorite()) {
            viewHolder.favoriteButton.setBackgroundResource(android.R.drawable.star_big_on);
        } else {
            viewHolder.favoriteButton.setBackgroundResource(android.R.drawable.star_big_off);
        }
        if (localDataSet.get(position).isWorking()) {
            viewHolder.workingView.setImageResource(android.R.drawable.presence_online);
        } else {
            viewHolder.workingView.setImageResource(android.R.drawable.presence_busy);
        }


        // Get element from your dataset at this position and replace the
        // contents of the view with that element

        viewHolder.favoriteButton.setOnClickListener(v -> {
            localDataSet.get(position).setFavorite(!localDataSet.get(position).getFavorite());
            updateChargingStation(position);
            notifyItemChanged(position);
        });


        viewHolder.navigationButton.setOnClickListener(l -> {
            String geoLocation = "google.navigation:q=" + localDataSet.get(position).getLatitude() + "," +
                    localDataSet.get(position).getLongitude();
            Uri gmmIntentUri = Uri.parse(geoLocation);
            Intent navigationIntent = new Intent(Intent.ACTION_VIEW, gmmIntentUri);
            navigationIntent.setPackage("com.google.android.apps.maps");
            mContext.startActivity(navigationIntent);

        });


    }

    @Override
    public int getItemCount() {
        return localDataSet.size();
    }

    private void updateChargingStation(int position) {
        NZSEDatabase db = NZSEDatabase.getDatabase(mContext);
        ChargingStationDAO dao = db.chargingStationDAO();
        Thread updateThread = new Thread(() -> dao.update(localDataSet.get(position)));
        updateThread.start();
        try {
            updateThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }


    public static class ViewHolder extends RecyclerView.ViewHolder {
        private final TextView addressTextView;
        private final TextView operatorTextView;
        private final ToggleButton favoriteButton;
        private final ImageButton navigationButton;
        private final TextView cityTextView;
        private final ImageView workingView;


        public ViewHolder(View view) {
            super(view);
            operatorTextView = view.findViewById(R.id.operatorMap);
            addressTextView = view.findViewById(R.id.addresMap);
            favoriteButton = view.findViewById(R.id.favoriteMap);
            cityTextView = view.findViewById(R.id.cityMap);
            workingView = view.findViewById(R.id.workingView);
            navigationButton = view.findViewById(R.id.navigationMap);

        }

    }
}
```

7.15 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/chargingStation.java File Reference

Classes

- class de.hda.nzse22.model.chargingStation

Packages

- package de.hda.nzse22.model

7.16 chargingStation.java

Go to the documentation of this file.

```
00001 package de.hda.nzse22.model;
00002
00003
00004 import androidx.annotation.NonNull;
00005 import androidx.room.ColumnInfo;
00006 import androidx.room.Entity;
00007 import androidx.room.PrimaryKey;
00008
00009 import java.util.List;
00010
00011
00012 @Entity(tableName = "chargingstations")
00013 public class chargingStation {
00014     @PrimaryKey(autoGenerate = true)
00015     public int uid;
00016
00017     @ColumnInfo(name = "Operator")
00018     public String mOperator = "";
00019
00020     @ColumnInfo(name = "Address")
00021     public String mAddress = "";
00022
00023     @ColumnInfo(name = "City")
00024     public String mCity = "";
00025
00026     @ColumnInfo(name = "County")
00027     public String mCounty = "";
00028
00029     @ColumnInfo(name = "Latitude")
00030     public float mLatitude = 0;
00031
00032     @ColumnInfo(name = "Longitude")
00033     public float mLongitude = 0;
00034
00035     @ColumnInfo(name = "MaxPower")
00036     public float mMaxPower = 0;
00037
00038     @ColumnInfo(name = "AmountOfChargingPoints")
00039     public Integer mAmountOfChargingPoints = 0;
00040
00041     @ColumnInfo(name = "KindofChargingStation")
00042     public String mKindofChargingStation = "";
00043
00044     @ColumnInfo(name = "Working")
00045     public boolean mWorking = true;
00046
00047     @ColumnInfo(name = "PlugType1")
00048     public String mPlugType1 = " ";
00049
00050     @ColumnInfo(name = "PlugType2")
00051     public String mPlugType2 = " ";
00052
00053     @ColumnInfo(name = "PlugType3")
00054     public String mPlugType3 = " ";
00055
00056     @ColumnInfo(name = "PlugType4")
00057     public String mPlugType4 = " ";
00058
00059     @ColumnInfo(name = "Favorite")
00060     public boolean isFavorite = false;
00061
```

```
    public boolean isExpand = false;

    public chargingStation(@NonNull List<String> ladestation) {
        this.mOperator = ladestation.get(0);
        this.mAddress = ladestation.get(1) + " " + ladestation.get(2);
        this.mCity = ladestation.get(4) + " " + ladestation.get(5);
        this.mCounty = ladestation.get(6);
        this.mLatitude = Float.parseFloat(ladestation.get(8).replace(',', '.'));
        this.mLongitude = Float.parseFloat(ladestation.get(9).replace(',', '.'));
        this.mMaxPower = Float.parseFloat(ladestation.get(11).replace(',', '.'));
        this.mKindofChargingStation = ladestation.get(12);
        this.mAmountOfChargingPoints = Integer.parseInt(ladestation.get(13));
        this.mPlugType1 = ladestation.get(14);
        if (ladestation.size() > 17) {
            this.mPlugType2 = ladestation.get(17);
        }
        if (ladestation.size() > 20) {
            this.mPlugType3 = ladestation.get(20);
        }
        if (ladestation.size() > 23) {
            this.mPlugType4 = ladestation.get(23);
        }
    }

    public chargingStation() {

    }

    public String getOperator() {
        return mOperator;
    }

    public String getAddress() {
        return mAddress;
    }

    public boolean isWorking() {
        return mWorking;
    }

    public void setWorking(boolean working) {
        this.mWorking = working;
    }

    public double getLatitude() {
        return mLatitude;
    }

    public double getLongitude() {
        return mLongitude;
    }

    public String getCity() {
        return mCity;
    }

    public boolean getFavorite() {
        return isFavorite;
    }

    public boolean isExpand() {
        return isExpand;
    }

    public void setExpand(boolean expand) {
        isExpand = expand;
    }

    public String getPlugType1() {
        return mPlugType1;
    }

    public int getUid() {
        return uid;
    }

    public void setUid(int uid) {
        this.uid = uid;
    }

    public String getmOperator() {
        return mOperator;
    }

    public String getmAddress() {
        return mAddress;
    }
```

```java

    public String getmCity() {
        return mCity;
    }

    public String getmCounty() {
        return mCounty;
    }

    public float getmLatitude() {
        return mLatitude;
    }

    public float getmLongitude() {
        return mLongitude;
    }

    public float getmMaxPower() {
        return mMaxPower;
    }

    public Integer getmAmountOfChargingPoints() {
        return mAmountOfChargingPoints;
    }

    public String getmKindofChargingStation() {
        return mKindofChargingStation;
    }

    public boolean ismWorking() {
        return mWorking;
    }

    public void setmWorking(boolean mWorking) {
        this.mWorking = mWorking;
    }

    public String getmPlugType1() {
        return mPlugType1;
    }

    public void setmPlugType1(String mPlugType1) {
        this.mPlugType1 = mPlugType1;
    }

    public String getmPlugType2() {
        return mPlugType2;
    }

    public void setmPlugType2(String mPlugType2) {
        this.mPlugType2 = mPlugType2;
    }

    public String getmPlugType3() {
        return mPlugType3;
    }

    public void setmPlugType3(String mPlugType3) {
        this.mPlugType3 = mPlugType3;
    }

    public String getmPlugType4() {
        return mPlugType4;
    }

    public void setmPlugType4(String mPlugType4) {
        this.mPlugType4 = mPlugType4;
    }

    public boolean isFavorite() {
        return isFavorite;
    }

    public void setFavorite(boolean favorite) {
        this.isFavorite = favorite;
    }


}
```

7.17 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/ChargingStationDAO.java File Reference

Classes

- interface de.hda.nzse22.model.ChargingStationDAO

Packages

- package de.hda.nzse22.model

7.18 ChargingStationDAO.java

Go to the documentation of this file.

```
package de.hda.nzse22.model;

import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;
import androidx.room.Update;

import java.util.List;

@Dao
public interface ChargingStationDAO {
    @Query("SELECT * FROM chargingstations")
    List<chargingStation> getAll();

    @Insert
    void insertAll(List<chargingStation> chargingStationList);

    @Update
    void update(chargingStation chargingStation);


    @Query("SELECT * FROM chargingstations WHERE Favorite=1")
    List<chargingStation> getFavorites();

    @Query("DELETE FROM chargingstations")
    void deleteAll();

    @Query("SELECT * FROM chargingstations WHERE Working=0")
    List<chargingStation> getNotWorkingChargingStations();

    @Query("SELECT * FROM chargingstations WHERE Latitude=:latitude AND Longitude=:longitude")
    List<chargingStation> getChargingStationsWithCoordinates(double latitude, double longitude);


}
```

7.19 D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/NZSEDatabase.java File Reference

Classes

- class de.hda.nzse22.model.NZSEDatabase

Packages

- package de.hda.nzse22.model

7.20 NZSEDatabase.java

Go to the documentation of this file.

```
00001 package de.hda.nzse22.model;
00002
00003 import android.content.Context;
00004
00005 import androidx.room.Database;
00006 import androidx.room.Room;
00007 import androidx.room.RoomDatabase;
00008
00012 @Database(entities = {chargingStation.class}, version = 4)
00013 public abstract class NZSEDatabase extends RoomDatabase {
00014     private static volatile NZSEDatabase INSTANCE;
00015
00021     public static NZSEDatabase getDatabase(final Context context) {
00022         if (INSTANCE == null) {
00023             synchronized (NZSEDatabase.class) {
00024                 if (INSTANCE == null) {
00025                     INSTANCE = Room.databaseBuilder(context.getApplicationContext(),
00026                                     NZSEDatabase.class, "chargingstation_database")
00027                             .build();
00028                 }
00029             }
00030         }
00031         return INSTANCE;
00032     }
00033
00039     public abstract ChargingStationDAO chargingStationDAO();
00040 }
```

Index

chargingStation
de.hda.nzse22.model.chargingStation, 7
chargingStationDAO
de.hda.nzse22.model.NZSEDatabase, 33
csvRead
de.hda.nzse22.download.GetChargingStationsActivity, 22
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/GetChargingStationsActivity.java, 43
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/download/RequestService.java, 46
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/FavoriteActivity.java, 48
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/favoritesAdapter/favoritesAdapter.java, 51
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MainActivity.java, 53
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/MapActivity.java, 56, 57
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/mapAdapter/mapAdapter.java, 59
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/chargingStation.java, 61
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/ChargingStationDAO.java, 64
D:/nzse_ss22/app/src/main/java/de/hda/nzse22/model/NZSEDatabase.java, 64, 65
de.hda.nzse22, 4
de.hda.nzse22.download, 4
de.hda.nzse22.download.GetChargingStationsActivity, 21
csvRead, 22
finish, 23
onCreate, 23
onRequestPermissionsResult, 24
de.hda.nzse22.download.RequestService, 35
FILENAME, 39
FILEPATH, 39
NOTIFICATION, 40
onBind, 37
onStartCommand, 37
RequestService, 37
REQUESTURL, 40
RESULT, 40
setCallback, 39
UNIQUEID, 40
de.hda.nzse22.download.RequestService.RequestServiceBinder, 41
setCallback, 42
de.hda.nzse22.FavoriteActivity, 16
onCreate, 17
onResume, 18
de.hda.nzse22.favoritesAdapter, 4
de.hda.nzse22.favoritesAdapter.favoritesAdapter, 18
favoritesAdapter, 19
getItemCount, 20
onBindViewHolder, 20
onCreateViewHolder, 20
de.hda.nzse22.MainActivity, 25
onCreate, 26
onDestroy, 26
onResume, 26
de.hda.nzse22.MapActivity, 27
onCreate, 28
onMapReady, 28
onResume, 29
de.hda.nzse22.mapAdapter, 4
de.hda.nzse22.mapAdapter.mapAdapter, 29
getItemCount, 31
mapAdapter, 30
onBindViewHolder, 31
onCreateViewHolder, 31
de.hda.nzse22.model, 4
de.hda.nzse22.model.chargingStation, 5
chargingStation, 7
getAddress, 7
getCity, 7
getFavorite, 7
getLatitude, 7
getLongitude, 7
getmAddress, 8
getmAmountOfChargingPoints, 8
getmCity, 8
getmCounty, 8
getmKindofChargingStation, 8
getmLatitude, 8
getmLongitude, 8
getmMaxPower, 8
getmOperator, 9
getmPlugType1, 9
getmPlugType2, 9
getmPlugType3, 9
getmPlugType4, 9
getOperator, 9
getPlugType1, 9
getUid, 9
isExpand, 10, 11
isFavorite, 10, 11
ismWorking, 10
isWorking, 10
mAddress, 12
mAmountOfChargingPoints, 12
mCity, 12
mCounty, 12
mKindofChargingStation, 12
mLatitude, 12
mLongitude, 12
mMaxPower, 12
mOperator, 13

mPlugType1, 13
mPlugType2, 13
mPlugType3, 13
mPlugType4, 13
mWorking, 13
setExpand, 10
setFavorite, 10
setmPlugType1, 10
setmPlugType2, 10
setmPlugType3, 11
setmPlugType4, 11
setmWorking, 11
setUid, 11
setWorking, 11
uid, 13
de.hda.nzse22.model.ChargingStationDAO, 14
deleteAll, 14
getAll, 14
getChargingStationsWithCoordinates, 15
getFavorites, 15
getNotWorkingChargingStations, 15
insertAll, 15
update, 16
de.hda.nzse22.model.NZSEDatabase, 32
chargingStationDAO, 33
getDatabase, 34
deleteAll
de.hda.nzse22.model.ChargingStationDAO, 14

favoritesAdapter
de.hda.nzse22.favoritesAdapter.favoritesAdapter, 19
FILENAME
de.hda.nzse22.download.RequestService, 39
FILEPATH
de.hda.nzse22.download.RequestService, 39
finish
de.hda.nzse22.download.GetChargingStationsActivity, 23

getAddress
de.hda.nzse22.model.chargingStation, 7
getAll
de.hda.nzse22.model.ChargingStationDAO, 14
getChargingStationsWithCoordinates
de.hda.nzse22.model.ChargingStationDAO, 15
getCity
de.hda.nzse22.model.chargingStation, 7
getDatabase
de.hda.nzse22.model.NZSEDatabase, 34
getFavorite
de.hda.nzse22.model.chargingStation, 7
getFavorites
de.hda.nzse22.model.ChargingStationDAO, 15
getItemCount
de.hda.nzse22.favoritesAdapter.favoritesAdapter, 20
de.hda.nzse22.mapAdapter.mapAdapter, 31
getLatitude
de.hda.nzse22.model.chargingStation, 7
getLongitude
de.hda.nzse22.model.chargingStation, 7
getmAddress
de.hda.nzse22.model.chargingStation, 8
getmAmountOfChargingPoints
de.hda.nzse22.model.chargingStation, 8
getmCity
de.hda.nzse22.model.chargingStation, 8
getmCounty
de.hda.nzse22.model.chargingStation, 8
getmKindofChargingStation
de.hda.nzse22.model.chargingStation, 8
getmLatitude
de.hda.nzse22.model.chargingStation, 8
getmLongitude
de.hda.nzse22.model.chargingStation, 8
getmMaxPower
de.hda.nzse22.model.chargingStation, 8
getmOperator
de.hda.nzse22.model.chargingStation, 9
getmPlugType1
de.hda.nzse22.model.chargingStation, 9
getmPlugType2
de.hda.nzse22.model.chargingStation, 9
getmPlugType3
de.hda.nzse22.model.chargingStation, 9
getmPlugType4
de.hda.nzse22.model.chargingStation, 9
getNotWorkingChargingStations
de.hda.nzse22.model.ChargingStationDAO, 15
getOperator
de.hda.nzse22.model.chargingStation, 9
getPlugType1
de.hda.nzse22.model.chargingStation, 9
getUid
de.hda.nzse22.model.chargingStation, 9
insertAll
de.hda.nzse22.model.ChargingStationDAO, 15
isExpand
de.hda.nzse22.model.chargingStation, 10, 11
isFavorite
de.hda.nzse22.model.chargingStation, 10, 11
ismWorking
de.hda.nzse22.model.chargingStation, 10
isWorking
de.hda.nzse22.model.chargingStation, 10

mAddress
de.hda.nzse22.model.chargingStation, 12
mAmountOfChargingPoints
de.hda.nzse22.model.chargingStation, 12
mapAdapter
de.hda.nzse22.mapAdapter.mapAdapter, 30
mCity
de.hda.nzse22.model.chargingStation, 12
mCounty
de.hda.nzse22.model.chargingStation, 12

mKindofChargingStation
de.hda.nzse22.model.chargingStation, 12
mLatitude
de.hda.nzse22.model.chargingStation, 12
mLongitude
de.hda.nzse22.model.chargingStation, 12
mMaxPower
de.hda.nzse22.model.chargingStation, 12
mOperator
de.hda.nzse22.model.chargingStation, 13
mPlugType1
de.hda.nzse22.model.chargingStation, 13
mPlugType2
de.hda.nzse22.model.chargingStation, 13
mPlugType3
de.hda.nzse22.model.chargingStation, 13
mPlugType4
de.hda.nzse22.model.chargingStation, 13
mWorking
de.hda.nzse22.model.chargingStation, 13

NOTIFICATION
de.hda.nzse22.download.RequestService, 40

onBind
de.hda.nzse22.download.RequestService, 37
onBindViewHolder
de.hda.nzse22.favoritesAdapter.favoritesAdapter, 20
de.hda.nzse22.mapAdapter.mapAdapter, 31
onCreate
de.hda.nzse22.download.GetChargingStationsActivity, 23
de.hda.nzse22.FavoriteActivity, 17
de.hda.nzse22.MainActivity, 26
de.hda.nzse22.MapActivity, 28
onCreateViewHolder
de.hda.nzse22.favoritesAdapter.favoritesAdapter, 20
de.hda.nzse22.mapAdapter.mapAdapter, 31
onDestroy
de.hda.nzse22.MainActivity, 26
onMapReady
de.hda.nzse22.MapActivity, 28
onRequestPermissionsResult
de.hda.nzse22.download.GetChargingStationsActivity, 24
onResume
de.hda.nzse22.FavoriteActivity, 18
de.hda.nzse22.MainActivity, 26
de.hda.nzse22.MapActivity, 29
onStartCommand
de.hda.nzse22.download.RequestService, 37

RequestService
de.hda.nzse22.download.RequestService, 37
REQUESTURL
de.hda.nzse22.download.RequestService, 40
RESULT
de.hda.nzse22.download.RequestService, 40

setCallback
de.hda.nzse22.download.RequestService, 39
de.hda.nzse22.download.RequestService.RequestServiceBinder, 42
setExpand
de.hda.nzse22.model.chargingStation, 10
setFavorite
de.hda.nzse22.model.chargingStation, 10
setmPlugType1
de.hda.nzse22.model.chargingStation, 10
setmPlugType2
de.hda.nzse22.model.chargingStation, 10
setmPlugType3
de.hda.nzse22.model.chargingStation, 11
setmPlugType4
de.hda.nzse22.model.chargingStation, 11
setmWorking
de.hda.nzse22.model.chargingStation, 11
setUid
de.hda.nzse22.model.chargingStation, 11
setWorking
de.hda.nzse22.model.chargingStation, 11

uid
de.hda.nzse22.model.chargingStation, 13
UNIQUEID
de.hda.nzse22.download.RequestService, 40
update
de.hda.nzse22.model.ChargingStationDAO, 16